

PVDetector: Pretrained Vulnerability Detection on Vulnerability-enriched Code Semantic Graph

JIAYUAN LI, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China and School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

LEI CUI, Guangxi Normal University, Guilin, China and Harbin Institute of Technology, Harbin, China

JIE ZHANG, LUN LI, RONGRONG XI, and HONGSONG ZHU, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China

Automated vulnerability detection is a critical issue in software security. The advent of Deep Learning (DL) has led to numerous studies employing DL to detect vulnerabilities in software source code. However, existing approaches still perform poorly, particularly with real-world vulnerabilities, due to the difficulty in accurately capturing their properties. To this end, we introduce PVDetector, a DL-based approach that utilizes rich code semantics, incorporates vulnerability knowledge, and leverages pretrained code representations for precise vulnerability detection. At its core, PVDetector employs a new model called Vulnerability-enriched Code Semantic Graph (VCSG), which accurately characterizes functions by distinguishing the semantics of identical variables and more finely capturing control dependencies, data dependencies, and vulnerability relationships. Additionally, we introduce four pretraining tasks specifically designed to learn the semantics of control, data, vulnerability, and variables from the VCSG model. These pretraining tasks significantly enhance PVDetector's capability to detect vulnerabilities in downstream tasks. Experimental results indicate that PVDetector outperforms SOTAs by 5.0–12.5% in precision, 0.2–9.7% in recall, and 3.0–15.1% in F1-score. Additionally, it supports six programming languages and demonstrates high efficiency (e.g., $10.6 \times$ faster than DeepDFA). When applied to seven software products, PVDetector discovered 55 vulnerabilities, including 10 silently patched flaws that had not been previously reported.

CCS Concepts: • **Security and privacy** → **Software and application security**;

Additional Key Words and Phrases: Vulnerability Detection, Code Semantic Graph, Pretrain-learning

ACM Reference format:

Jiayuan Li, Lei Cui, Jie Zhang, Lun Li, Rongrong Xi, and Hongsong Zhu. 2026. PVDetector: Pretrained Vulnerability Detection on Vulnerability-enriched Code Semantic Graph. *ACM Trans. Softw. Eng. Methodol.* 35, 6, Article 157 (May 2026), 31 pages.

<https://doi.org/10.1145/3768582>

This work is supported by the Network Twinning Enhancement Methods project (E3Z0111104) from the Institute of Information Engineering, Chinese Academy of Sciences.

Authors' Contact Information: Jiayuan Li, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China and School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China; e-mail: lijia yuan@iie.ac.cn; Lei Cui (corresponding author), Guangxi Normal University, Guilin, China and Harbin Institute of Technology, Harbin, China; e-mail: cuilei@zgclab.edu.cn; Jie Zhang, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China; e-mail: zhangjie@iie.ac.cn; Lun Li, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China; e-mail: lilun@iie.ac.cn; Rongrong Xi (corresponding author), Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China; e-mail: xirongrong@iie.ac.cn; Hongsong Zhu, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China; e-mail: zhuhongsong@iie.ac.cn.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).

ACM 1557-7392/2026/5-ART157

<https://doi.org/10.1145/3768582>

1 Introduction

Software vulnerabilities provide a common starting point for many cyber attacks. The increasing number of vulnerabilities reported in the **Common Vulnerabilities and Exposures (CVE)** database [15] highlights the growing threat to cyberspace [28, 29, 60]. Therefore, automated methods for detecting vulnerabilities are crucial to mitigate this expanding attack surface.

Vulnerability detection techniques are broadly categorized into static analysis [20, 37, 81, 95, 98] and dynamic analysis [5, 8, 33, 46]. Static analysis examines software code directly without execution, offering scalable and automated detection. Thus, this article focuses on static analysis at the function level for source code, commonly available on platforms like GitHub. Static analysis include code similarity-based, rule-based, and **Machine Learning (ML)**-based methods. Code similarity-based methods [14, 31, 34, 65, 94] primarily detect vulnerabilities from code cloning but often miss non-cloning bugs. Rule-based methods [10, 67, 91] rely on manually defined rules and struggle with new vulnerabilities that do not exhibit existing patterns. ML-based [25, 53, 97] and **Deep Learning (DL)**-based methods [21, 24, 40, 41, 47, 51, 56, 57, 62, 73, 79, 90, 93] automatically extract features from raw code or abstracted code structures to enhance vulnerability detection.

1.1 Problems

Deep Learning-based Vulnerability Detection (DLVD) has gained popularity in recent years, adapting methods from natural language processing to source code analysis. For example, VulDeeP-ecker employs a bidirectional **Long Short-Term Memory (LSTM)** network to extract features from code gadgets for vulnerability detection [49]. Subsequent approaches integrate **Convolutional Neural Networks (CNN)** and LSTM with attention mechanisms to enhance detection capabilities [48, 64, 103]. However, these text-based methods often achieve sub-optimal performance as they simply treat source code as natural language, overlooking its inherent logical semantics. To address this, later studies use program analysis techniques to extract syntactic and semantic information from source code, creating structured code representations from **Abstract Syntax Tree (AST)**, **Control Flow Graph (CFG)**, **Data Flow Graph (DFG)**, and **Code Property Graph (CPG)** [11, 17, 92, 95, 101] and then utilize DL models to detect vulnerabilities [9, 21, 90, 102].

However, existing DLVD methods still face performance limitations in real-world scenarios due to the complexity of code semantics [9]. First, they fail to extract comprehensive information, treating variables uniformly even when their semantics differ across usages (e.g., in a loop). Second, they primarily focus on control and data dependencies between statements, often overlooking within-statement dependencies like assignments (e.g., $a = a + b$) and memory operations (e.g., *strcpy*). Third, they do not incorporate specific vulnerability information, relying on broad representations (e.g., DFG) of the entire function instead of targeting vulnerable code snippets. Additionally, DL models such as CNN or LSTM suffer limited feature extraction capabilities. Although some methods [56, 62] utilize code pretrained models [23, 26], they focus on enabling models to correctly learn the features without adapting them to logically structured codes and vulnerabilities. Concurrently, methods [51, 57, 76] employing **Large Language Models (LLMs)** have also revealed limitations [79], including high **False-Positive (FP)** rates, non-deterministic results, and substantial computational costs.

1.2 Our Solutions

Following expert practices in detecting vulnerabilities, we identify three critical aspects of DL-based vulnerability detection: code semantics acquisition, vulnerability knowledge utilization, and DL-based feature extraction. Motivated by this, we introduce PVDetector to automatically and precisely identify vulnerabilities in source code. PVDetector consists of two main components. The first is a graph-based representation called the **Vulnerability-enriched Code Semantic**

Graph (VCSG), which captures precise code semantics and highlights potential security issues. The second is a pretraining strategy named Semantics Pretraining, which introduces four specialized pretraining tasks on the VCSG model to enhance DL-based feature extraction, thereby improving the semantic understanding of function code.

The VCSG models a function through three interrelated components: a semantics-aware dataflow that captures the different meanings behind variables, a condition-sensitive control flow that reflects execution logic under varying conditions, and an embedded representation of vulnerability knowledge to highlight potential security risks. First, VCSG distinguishes between the semantics of the same variable within a function. For example, in operation $a = a + b$, it differentiates the left-side a from the right-side a , enabling a more precise depiction of dataflow. This distinction is crucial for identifying vulnerabilities stemming from changes in variable values. Similar approaches have also been explored in the domain of OS code bug-hunting [43, 44, 72], yielding excellent results. This further demonstrates the advantage of applying semantic distinction in DL-based vulnerability detection. Second, VCSG introduces a condition-aware control flow by breaking down the conditional statement into an identifier (e.g., *if*) and two expressions (raw condition and its negation) and then linking these components with subsequent branches based on the outcomes (True or False). This refinement enhances control semantics with specific operations and their causes. Third, we have compiled a vulnerability knowledge base by analyzing around 600 vulnerabilities of 25 CWEs. By representing each entry with node dependencies in VCSG, the model can capture detailed vulnerability semantics, significantly improving the vulnerability detection performance.

Building upon the VCSG model, we introduce a pretraining strategy named Semantics Pretraining. This strategy is carefully aligned with the components of VCSG, comprising four specialized tasks: (1) A **Data Flow Prediction (DFP)** task to capture data dependency through variable assignments and function calls; (2) A **Control Flow Prediction (CFP)** task to model how conditional branches and control structures influence code behavior; (3) A **Vulnerability Knowledge Prediction (VKP)** task to embed prior security knowledge into the representation space; (4) A **Variable Semantic Distinction (VSD)** task to enhance the understanding of variable usage and semantics. Together, these tasks guide the model in learning rich, vulnerability-aware semantic representations from the VCSG.

We evaluate PVDetector on five datasets and compare it against 10 **State-of-the-Art (SOTA)** DLVD methods [9, 24, 48, 49, 56, 62, 73, 90, 93, 102]. PVDetector outperforms the best SOTA method by 5.0–12.5% in precision, 0.2–9.7% in recall, and 3.0–15.1% in F1-score. In real-world scenarios, PVDetector identifies 55 vulnerabilities, 10 of which are not reported in the NVD [58]. Additionally, we evaluate PVDetector’s effectiveness across six programming languages and also confirm that VCSG can enhance the vulnerability detection capability of four LLMs.

To summarize, this article makes the following contributions:

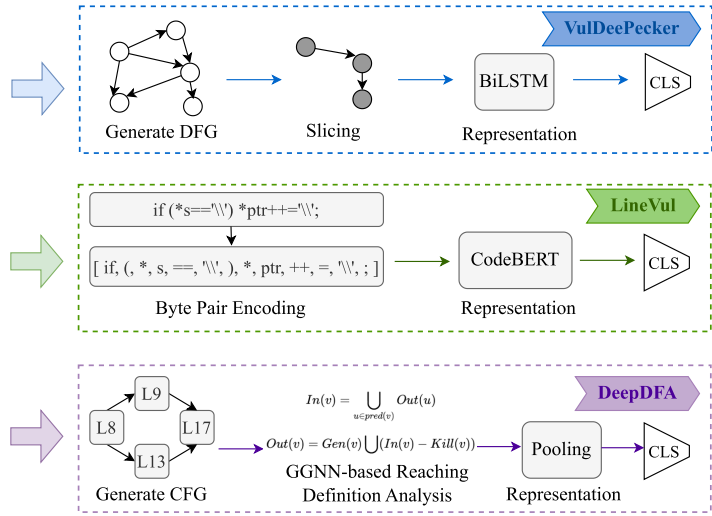
- First, a new DL-based vulnerability detection approach, PVDetector, utilizes rich code semantics, vulnerability knowledge, and pretrained tasks for enhanced detection (Section 2).
- Second, a new graph model, VCSG, more finely characterizes code semantics by differentiating different semantics of the same variables and refining control flow based on conditional outcomes. Additionally, VCSG incorporates vulnerability knowledge to enhance the understanding and exposure of vulnerabilities (Section 3).
- Third, a pretraining method, Semantics Pretraining, designs four pretraining tasks to learn control, data, vulnerability, and variable semantics from the VCSG model. This facilitates more effective detection of vulnerabilities (Section 4).
- Last, implementation (Section 5) and evaluation (Section 6) of PVDetector. The experimental results demonstrate that PVDetector significantly outperforms previous SOTA methods in vulnerability detection. Additionally, it can generalize to six programming languages.

```

void process_args(int mode, char** argv)
{
  1 size_t total_size = 0;
  2 for (char** av = argv; *av; av++) {
  3   total_size += strlen(*av) + 1;
  4 }
  5 char* buffer = malloc(total_size);
  6 char* ptr = buffer;
  7 for (char** av = argv; *av; av++) {
  8   if (mode == SHELL_MODE) {
  9     for (char* s = *av; *s; s++) {
 10       if (*s == '\\') *ptr++ = '\\';
 11       *ptr++ = *s;
 12     }
 13   } else {
 14     memcpy(ptr, *av, strlen(*av));
 15     ptr += strlen(*av);
 16   }
 17   *ptr++ = '\0';
 18 }
 19 ptr[-1] = '\0';
}

```

(a) Motivation example



(b) Representative DLVD approaches

Fig. 1. Three representative DLVD approaches for detecting vulnerability.

2 Basic Design

2.1 Limitations of Existing Approaches

We use the real-world vulnerability CVE-2021-3156 as a motivating example to demonstrate the limitations in existing vulnerability detection methods and highlight the core insight of our approach. Given space constraints, Figure 1(a) presents a simplified version.

This vulnerability is a heap-based buffer overflow, stemming from the improper handling of escape sequences for the backslash character during command-line argument processing. Specifically, when the program operates in *SHELL_MODE*, it performs escape processing on backslash characters within input arguments. This leads to a mismatch between the actual number of bytes written and the initially computed *total_size*, resulting in buffer overflow. An attacker can exploit this vulnerability by crafting excessively long arguments containing a large number of backslashes, thereby triggering the overflow. To detect this vulnerability, a detection model must satisfy the following requirements: (1) distinguish the semantic differences of the same variable (e.g., **ptr*) across different stages of execution; (2) accurately capture both data and control dependencies related to such as **ptr* and **av*; (3) understand that the volume of data written via **ptr* must not exceed the initially allocated space denoted by *total_size*. Based on these observations, we analyze three representative DLVD approaches and discuss their limitations.

VulDeePecker [49] represents function with DFG and generates code snippets through data dependency slicing on identified vulnerability keywords (e.g., *strcpy*) and then employs LSTM to learn features for detection. Although VulDeePecker incorporates some vulnerability knowledge, it misses control dependencies during snippets generation, which are critical for detecting vulnerabilities involving conditional checks (e.g., constraints of **s* in line 10 of Figure 1(a)). Additionally, LSTM struggles to capture long-distance dependencies [3], reducing its effectiveness in feature extraction.

LineVul [24] leverages DL techniques such as byte pair encoding, multi-head attention, and pretrained models to obtain code representations. Specifically, it utilizes the CodeBERT [23] model, pretrained on a 20GB code corpus. While CodeBERT demonstrates powerful DL-based feature

extraction capabilities, LineVul lacks a thorough exploration of the code syntactic and semantic structures (i.e., AST, DFG), which impact the detection performance. For instance, in line 10 and line 11 of Figure 1(a), the two variables **ptr* would be encoded as identical embeddings by LineVul, resulting in imprecise data semantics and therefore sub-optimal detection performance.

DeepDFA [73] applies reaching definition dataflow analysis based on the CFG, constructing abstract dataflow embeddings by focusing on elements such as variable types, operators, and so on. Then, it utilizes DL models to update dataflow and the pooling layer to generate representation. Although DeepDFA offers comprehensive code semantics, it lacks vulnerability-specific insights, making it challenging to capture vulnerability patterns independent of elements such as variable types and operators. Taking the motivation example in Figure 1(a) as an illustration, DeepDFA's dataflow analysis fails to specify what constraints should be satisfied by **ptr* in line 6 and *total_size* in line 5 to prevent vulnerabilities. Merely analyzing the data dependency relation between **ptr* and *total_size* presents significant challenges for precisely localizing this buffer overflow vulnerability.

In summary, existing DLVD methods either lack precision in analyzing data and control dependencies or fail to incorporate vulnerability-related insights, making them unaware of the specific conditions under which such a vulnerability may occur. Therefore, an effective approach must be capable of accurately understanding code semantics while also embedding a certain level of vulnerability-specific knowledge, enabling it to identify the conditions under which vulnerabilities are likely to manifest and thereby achieve accurate vulnerability detection.

2.2 Basic Idea of PVDetector

Motivated by the aforementioned analysis, we propose PVDetector, designed to mimic the detection procedure by security experts (Figure 2(a)). An expert typically analyzes function code by understanding its semantics and leveraging empirical vulnerability knowledge to perform a comprehensive analysis for identifying vulnerabilities. Inspired by this, PVDetector introduces VCSG for enhanced code semantic acquisition and vulnerability knowledge utilization. Additionally, it incorporates Semantics Pretraining for feature extraction (Figure 2(b)). To better illustrate each component in PVDetector, we select the source code shown in Figure 2(b) to demonstrate our approach. This source code has a buffer overflow vulnerability in line 13, when *strcpy* stores a 40-byte input string (*temp*) into a memory region (*mem*) allocated for only 30 bytes (80–50).

VCSG offers more precise and comprehensive code semantics for a function. It enhances existing code graph models in three aspects: semantics-cognizant dataflow, condition-aware control flow, and vulnerability knowledge incorporation. First, VCSG captures data dependencies between variables across or within statements, enabling more accurate, semantics-cognizant dataflow representation. For example, in line 6 of Figure 1(a), the variable *mem_num* in *mem_num=mem_num-50* has different values on the left and right sides. VCSG treats *mem_num* as two distinct variables (i.e., *mem_num1* and *mem_num2*) with a data dependency between them, ensuring precise and unambiguous variable semantics throughout the model. Second, VCSG refines control flow by decomposing each conditional statement into more granular components, explicitly generating both the condition and its negation and connecting them to their respective branches, thereby differentiating the constraint conditions corresponding to different branches of conditional statements. This approach yields more precise control semantics in VCSG, as illustrated by the orange rectangles in Figure 2(b). Third, VCSG incorporates 20 vulnerability knowledge entries covering 25 popular CWEs. By representing each entry with nodes and edges, VCSG provides a more comprehensive view of the code's behavior and security posture. For example, in line 13 of Figure 1(a), for *strcpy* that is prone to buffer overflow, a "<" node is added from the source to the destination node (blue nodes in Figure 2(b)). This indicates that overflow occurs due to improper size checking (i.e., *dst < src*). Additionally, we propose an automated vulnerability knowledge generation method for subsequent extensibility.

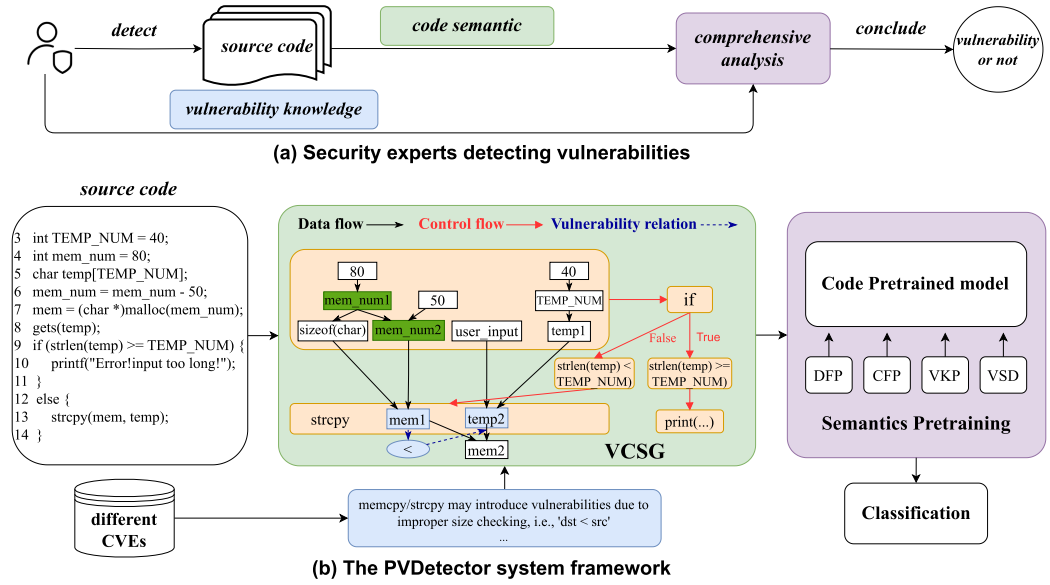


Fig. 2. The overview of PVDetector (b). It mimics expert vulnerability detection process (a) with a VCSG model (for code semantic acquisition and vulnerability knowledge utilization), and Semantics Pretraining (for code understanding and analysis).

Semantic Pretraining (SP) aims to enhance the code pretrained models (e.g., CodeBERT) with the ability to recognize and analyze the code semantics and vulnerability relations in VCSG. We design four specific pretraining tasks: DFP, CFP, VKP, and VSD, aiming to align the nodes in VCSG with the corresponding tokens in source code, therefore enabling a deeper understanding of the VCSG semantics. After the pretraining stage, the semantic-pretrained CodeBERT (i.e., we call it SP-BERT) not only maintains its original capability to comprehend source code but also extends its functionality to understand VCSG which encapsulates rich semantics.

2.3 Novelty of Our Approach

PVDetector is a new DL-based method that differs from existing methods in two key aspects. First, it proposes a VCSG model that provides richer and more fine-grained code semantics. For dataflow, existing graph models either crudely fail to distinguish between variable states despite their value being modified (e.g., DFG, CPG), or excessively label them all as different even though their value remains the same [27]. In contrast, VCSG provides accurate data semantics by distinguishing the same variable only when it has different meanings. Meanwhile, unlike existing path-sensitive algorithms[12] that focus on specific execution paths, VCSG offers a DL model-oriented innovation by refining control flow based on conditional nodes of various branches, which enables DL models to extract more accurate control semantics. Additionally, VCSG uses identifiers and their relationships to represent the knowledge of a type of vulnerability at a fine-grained level, which differs from vulnerability knowledge graph [75, 87] that systematically captures and organizes information about vulnerabilities. This incorporation effectively enhances the understanding of vulnerabilities.

Second, it designs four specific pretraining tasks to learn the semantics of data, control, vulnerability, and variables. Unlike the existing pretraining-based methods [24, 56, 62] that primarily emphasize source code understanding, these tasks focus on capturing the intricate relationships

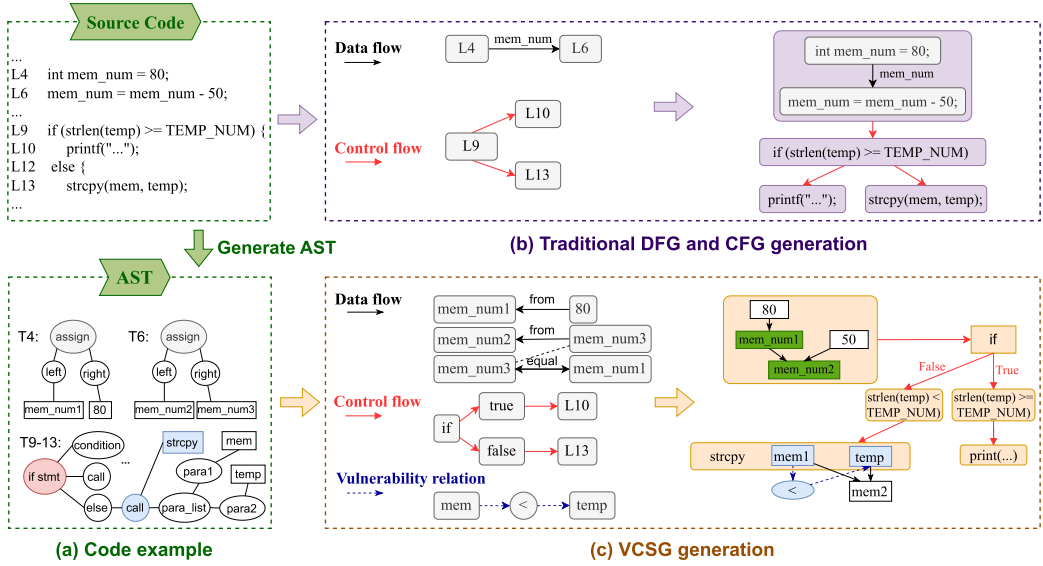


Fig. 3. Comparison between traditional DFG and CFG (b) with our proposed VCSG model (c).

between source code and VCSG, offering a more comprehensive and nuanced understanding of the semantics in VCSG, thus enhancing vulnerability detection capabilities.

3 VCSG Model

3.1 Overview

The VCSG generation process is illustrated in Figure 3(c). In contrast to the traditional DFG and CFG shown in Figure 3(b), VCSG introduces significant enhancements to both dataflow and control flow representations. Specifically, it proposes the concepts of semantics-cognizant dataflow and condition-aware control flow. Furthermore, VCSG incorporates vulnerability knowledge to extract potential vulnerability relationships between variables and statements within the code. These innovations aim to capture a more comprehensive and accurate representation of code semantics. Taking the code in Figure 3(a) as an example (the key part of the source code in Figure 2(b)), the following sections will provide a detailed description of each component of the VCSG model.

3.2 Semantics-Cognizant Dataflow

Traditional dataflow typically delineates relationships between two statements concerning the same variable, as shown in Definition 1. However, the same variable can have different semantics across or within statements due to the value mutations. For example, in `mem_num = mem_num - 50` on line 6 in Figure 3(a), the left-side `mem_num` denotes a value 50 less than the right one. If these differences are not recognized, the DL model will treat them as identical, as shown in Figure 3(b). To address this, we propose a semantics-cognizant dataflow model, which captures fine-grained dataflow between variables both within a single statement and across statements, as described in Definition 2.

Definition 1 (Traditional Dataflow) [48]. Given a function composed of statements, $F = \{f_1, \dots, f_n\}$. For a variable v belonging to statement f_i , if v is also referred in f_j ($j \neq i$), then there is a dataflow from statement f_i to f_j on token v .

Definition 2 (Semantics-Cognizant Dataflow). Given a function composed of variables, $F = \{v_1, \dots, v_m\}$. For two variables $v_i, v_j \in F$ (i can be equal to j), if the value of v_j is derived from v_i , then there exists a dataflow from v_i to v_j .

Following Definition 2, *mem_num1* and *mem_num2* nodes are created, and an edge is added between them in VCSG, as shown by green rectangles in Figure 3(c). This renaming ensures consistent semantics for variables with the same name within VCSG, and it allows the DL model to distinguish between semantically different variables. Thus, it offers more precise data semantics.

Note that although the semantic-cognizant dataflow operates at the token level and thus may lack certain semantic information beyond variables compared to traditional line-level data flows, this missing information is complemented within the condition-aware control flow. Consequently, VCSG's ability to capture the vulnerability-related context remains unaffected.

3.3 Condition-Aware Control Flow

In traditional CFGs, conditional statements are typically represented by a parent node with two (occasionally one) child nodes corresponding to the true and false branches. Although this format is suitable for static analysis in many scenarios, it poses significant challenges for DL models. Specifically, as both the true and false branch nodes connect to the same condition node, it fails to differentiate the unique contextual dependencies of each branch, which can hinder the model's ability to learn the function's control semantics accurately. For example, as illustrated in Figure 3(b), both line 10 and line 13 are connected to the node representing line 9. From the perspective of DL models, the constraints on the *temp* variable in both line 10 and line 13 statements appear to be $strlen(temp) \geq TEMP_NUM$. This can lead to the model learning incorrect constraint conditions, consequently impacting the effectiveness of vulnerability detection.

To address this issue, we propose a condition-aware control flow model, as illustrated in Figure 3(c). This model decomposes each conditional statement and represents the conditional check and its logical outcomes as distinct nodes. For the above example, the *if* node serves as the parent node with two child nodes representing the condition ($strlen(temp) \geq TEMP_NUM$) and its negation ($strlen(temp) < TEMP_NUM$). The condition node connects to the true branch, while the negation node connects to the false branch (line 13). The explicit separation of conditions and their outcomes provides a more detailed and unambiguous representation of control logic. This granularity enables a finer understanding of how different logical paths are taken within the program. Although independently calculating the conditional constraints for each variable may appear more precise, the complexity introduced by this operation can hinder the DL model's ability to learn control semantics. This issue becomes more pronounced when detecting vulnerabilities in code with lengthy and complex execution paths. Therefore, we treat conditional statements as a whole, thereby avoiding the problem of excessive branching. By distinguishing between these paths, it becomes easier to identify edge cases that might lead to security issues.

3.4 Vulnerability Knowledge Utilization

To effectively leverage vulnerability knowledge and enrich the vulnerability semantics within the VCSG model, we identify three key challenges: (i) How can vulnerability knowledge be formally represented in VCSG? (ii) Given a specific CWE, how can we generate the corresponding formalized vulnerability knowledge in an automated and scalable manner? and (iii) Given the source code and vulnerability knowledge, how can we determine whether the knowledge is applicable to the code, and subsequently transform it into vulnerability relationships between VCSG nodes? The following sections will elaborate on our approaches to address these three challenges.

3.4.1 Vulnerability Knowledge Formulation. For the first challenge, based on syntactic levels of code, we design a three-layer paradigm to enable a step-by-step formulation of vulnerability knowledge.

Layer 1: Basic Formulas for a Single Variable. We establish four basic formulas according to a variable's basic operations throughout its lifecycle. Defined with edges and nodes, these formulas facilitate incorporation into VCSG. Let VAR denote the associated variable node and $\rightarrow$ denote the edge in VCSG. The formulas and their operations in VCSG are defined as follows:

Formula 1. Declare, represents the creation of the variable. In VCSG, it involves first creating a NULL node and then adding an edge from NULL to VAR (NULL $\rightarrow$ VAR).

Formula 2. Read, denotes the read operation on a variable. VCSG involves adding an edge originating from VAR (i.e., VAR $\rightarrow$). The destination node will be added in Layer 2.

Formula 3. Write, denotes the write operation on a variable. It adds an edge directed toward VAR (i.e., $\rightarrow$ VAR).

Formula 4. Destroy, represents the destruction of a variable, particularly for pointers. It creates a NULL node and links it to VAR (VAR $\rightarrow$ NULL) in VCSG.

Layer 2: Formulas for a Statement. Building on four basic formulas, this layer formalizes vulnerability knowledge for one statement. We design formulas for two cases: a single variable and two variables. A statement with more variables can be formalized by combining these two cases.

Single Variable. It provides specificity for read (*print(v)*) and write operations (*v=10*), i.e., DST or SRC node is created and connected to VAR, i.e., VAR $\rightarrow$ DST or SRC $\rightarrow$ VAR.

Two Variables. We consider three scenarios based on variable operations. (a) One read and one write. This involves value transfer like assignment or memory copy. To represent potential vulnerabilities when destination is smaller than source, we add a node " $<$ " and connect it to variable nodes, i.e., DST $\rightarrow<\rightarrow$ SRC. (b) Two reads. This covers operations like arithmetic, comparison, or memory access. We add an operation node and link it to variable nodes, e.g., VAR1 $\rightarrow+$, VAR2 $\rightarrow+$ for "*a+b*." (c) Two writes. Although rarely encountered, it can be handled similarly to Two reads.

Layer 3: Formula for Multiple Statements. For two statements, STMT1 and STMT2, sharing the same variable, we add two edges in VCSG: one from STMT1 to the variable and another from the variable to STMT2. Multiple statements can be handled by combining pairs of statements.

It is worth mentioning that the direction of the arrow does not have a consistent meaning. Sometimes it indicates the change of the variable state (such as VAR $\rightarrow$ NULL), and sometimes it indicates specific semantics through the order of parent node and child node (such as DST $\rightarrow<\rightarrow$ SRC). The actual meaning can be learned by the DL model during training.

3.4.2 Vulnerability Knowledge Generation. For the second challenge, we have designed an automated process for vulnerability knowledge generation. For a specific CWE, we first prepare the diffs between vulnerabilities and patches of this CWE. Next, it symbolizes the diff codes and extracts frequently appearing snippets using the Apriori algorithm [1]. Finally, for each snippet involving one or multiple statements, the vulnerability knowledge is generated following the aforementioned formulas. It is worth mentioning that personal experience can also be combined with these formulas in the generation process. For example, for vulnerabilities involving access to sensitive information (e.g., *var="pwd123"*, CWE-798), one can add a PWD node and connect it to VAR, i.e., PWD $\rightarrow$ VAR. For buffer overflow (e.g., CWE-119), caused by improper size checking when the size of *dst* is smaller than *src* in *strcpy(dst, src)*, a knowledge entry "*dst $\rightarrow<\rightarrow$ src*" can be generated. We have compiled a vulnerability knowledge base containing 20 entries by analyzing around 600 CVEs from the CWE Top 25 Most Dangerous Software Weaknesses List [16]. These 20 entries can be expanded to cover a wider range of vulnerability knowledge following the aforementioned process, which we leave as a direction for future work. Due to the page limits, we present three entries in Table 1.

Table 1. Samples of Vulnerability Knowledge in VCSG

Category	Snippets	Formulation	Keywords
Buffer Overflow (CWE119/120/121/122)	<code>strcpy(dst,src);</code>	$dst \rightarrow < \rightarrow src;$	<code>strcpy, memcpy</code>
Use After Free (CWE416)	<code>free(p);</code> <code>p = q;</code>	$NULL \rightarrow p; q \rightarrow p;$ $free(p) \rightarrow p \rightarrow p = q;$	<code>free</code>
Uninitialized Variable (CWE457)	<code>int a;</code> <code>b = a + 1;</code>	$NULL \rightarrow a \rightarrow +; a + 1 \rightarrow b;$ $int\ a \rightarrow a \rightarrow b = a + 1;$	<code>int, char</code>

3.4.3 Vulnerability Knowledge Incorporation. For the third challenge, we identify the keywords for each vulnerability knowledge, as illustrated in Table 1. If a code snippet contains vulnerability keywords corresponding to a specific piece of vulnerability knowledge, it indicates that this knowledge is applicable to the given code segment. In such cases, the corresponding VCSG node is augmented with new nodes and edges following the formulation mentioned above to represent the potential issues. In the sample function, for the vulnerability keyword *strcpy*, a new node “<” is introduced from *mem1* to *temp2* following $dst \rightarrow < \rightarrow src$, as shown in Figure 3(c).

3.4.4 Summary. Through the formulation, generation, and incorporation of vulnerability knowledge, we successfully address the three aforementioned challenges and enrich the vulnerability semantics within the VCSG model. For a source code under detection, we first identify vulnerability keywords as summarized in Section 3.4.3. When a particular vulnerability knowledge is deemed applicable to the code, we incorporate the previously formalized vulnerability knowledge (in the form of nodes and edges in the VCSG) into the VCSG, thereby enriching the vulnerability code semantics. As described above, if one wishes to consider vulnerability knowledge corresponding to any new vulnerability type, it can be generated automatically and added to the vulnerability knowledge base through this methodology. This confers significant scalability to our approach, enabling other researchers or engineers to expand upon it according to their specific requirements.

3.5 VCSG Generation Algorithm

To generate the VCSG graph, we first parse the source code of a function into an AST, as shown in Figure 3(a). Each AST node is then examined to derive data and control flow based on its type, and Algorithm 1 presents the pseudocode for this process. For example, an initialization statement yields dataflow from the value to the variable, while an assignment statement generates dataflow from the right-side to the left-side variables. Certain specialized functions may exhibit unique dataflow relationships, such as memory functions, the semantics of variables before and after memory write operations can undergo significant changes. We also incorporate these dataflows into our consideration. For conditional statements, control flow originates from the identifier node (e.g., *if*) to the condition (or negation) node and then to the associated block. After constructing the control and dataflows, we incorporate the vulnerability knowledge and generate the vulnerability relations using the method in Section 3.4.3. Finally, this process results in a VCSG that accurately captures rich code semantics, which can be used for more effective vulnerability detection.

Algorithm 1: VCSG Generation Algorithm

Input: F: Source code of a function
Output: VCSG

```

1 AST ← ASTParsing(F) ;
2 Node_list ← DFS(AST.root_node) ;
3 Sc_data_flow ← {}, Ca_control_flow ← {}, Vul_relation ← {} ;
4 foreach node ∈ Node_list do
5     switch node.type do
6         case init_statement do
7             ADD "var ← value" to Sc_data_flow ;
8             ADD "NULL → var" to Vul_relation;
9         case assign_statement do
10            ADD "left ← right" to Sc_data_flow ;
11            ADD "var → op" to Vul_relation ;
12        case if_statement do
13            ADD "if → condition → if_body" to Ca_control_flow ;
14            ADD "if → negation → else_body" to Ca_control_flow ;
15        case call_statement do
16            switch call_statement.function_name do
17                case memory_function do
18                    ADD "dst' ← src" to Sc_data_flow;
19                    ADD "dst' ← dst" to Sc_data_flow;
20                    ADD "dst → <→ src" to Vul_relation;
21                ...
22            end
23        ...
24    end
25 end
26 VCSG ← Merge(Sc_data_flow, Ca_control_flow, Vul_relation) ;
27 return VCSG;

```

4 Semantics Pretraining

Pretraining enables a comprehensive understanding of data. Existing code pretrained models like CodeBERT learn token embeddings based on nearby tokens in sequential raw code; however, we aim for tokens to acquire various semantics from the structured VCSG model. Thus, we design four pretraining tasks to thoroughly capture the semantics of data, control, vulnerability, and variables.

4.1 DFP

The DFP task is designed to acquire data semantics for tokens from VCSG, enhancing PVDetector's understanding of data dependencies. It predicts whether a specified flow constitutes a dataflow for given statements, formulated as (*Stmts*, [SEP], *Flow*). This enables the model to recognize that tokens in *Stmts* are involved in dataflow, thereby obtaining data semantics. Figure 4(a) shows a sample with statements from lines 3 and 5 and a dataflow from 40 to *temp*, allowing the model to learn *temp*'s semantics related to value 40. Similarly, it learns the data semantics of *mem* related to

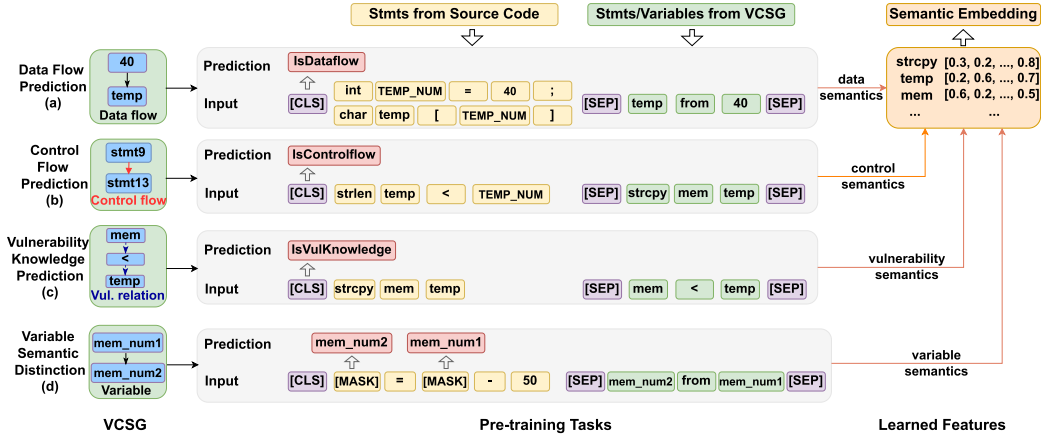


Fig. 4. Four pretraining tasks on source code and VCSG to learn various semantics.

30 (i.e., 80–50) through *Stmts* in lines 4, 6, and 7 and the *Flow* involving *mem*. Subsequently, the model can learn the semantics of these two variables in terms of their values.

Task Formulation. Let DP denote the dataflows existing in VCSG and SP denote the synthetic flows not present in VCSG. For a flow d_i , $s_i || d_i$ ($d_i \in DP$) denotes a positive sample where s_i denote the associated statements, and “||” signifies concatenation. Similarly, $s_i || d_i$ ($d_i \in SP$) denotes a negative sample. This task aims to minimize the binary cross-entropy loss, defined as:

$$L_{DFP} = - \sum_{d_i \in DP \cup SP} (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)), \quad (1)$$

where y_i is the true label of $s_i || d_i$, $\hat{y}_i$ is the predicted probability that the sample is positive. $\hat{y}_i$ is computed by $\text{softmax}(O_{[CLS]}(S || d_i))$, where S denotes the source code and $O_{[CLS]}$ is the output value of the [CLS] token in the final layer of the DFP task.

4.2 CFP

This CFP task captures control semantics from VCSG. Pretrained models like word2vec and BERT, which analyze code sequentially, struggle with complex control semantics due to the non-linear nature of code execution. To address this, we utilize the Next Sentence Prediction task in BERT, predicting whether $Stmt_1$ and $Stmt_2$ exhibit a control dependency, formatted as $(Stmt_1, [SEP], Stmt_2)$. In a positive sample, $Stmt_2$ is the branch block following the conditional statement $Stmt_1$. Figure 4(b) illustrates a positive sample for a control flow from $Stmt_9$ to $Stmt_{13}$. This helps the model understand that *strcpy* depends on a preceding size check, where *temp* is involved. By combining control semantics with the data semantics of *temp* and *mem* (values 40 and 30, respectively), PVDetector can more accurately analyze how the size of *mem* changes and affects the *strcpy* operation, aiding in identifying buffer overflow vulnerability.

Task Formulation. The CFP task is similar to the DFP task but differs in sample construction. The CFP task forms positive samples using statements corresponding to control flow nodes, while the DFP task uses both statements and variables involved in dataflow.

4.3 VKP

The VKP task helps PVDetector acquire crucial vulnerability insights. Similar to DFP, it predicts whether *Vul* denotes the corresponding vulnerability relation for statements *Stmts*, formatted as $(Stmts, [SEP], Vul)$. As shown in Figure 4(c), for the relation “*mem* to < and then to *temp*,” we

generate a positive VKP sample, where line 13 (i.e., `strcpy(mem, temp)`) denotes *Stmts* and the relation represents *Vul*. This setup allows the model to incorporate vulnerability semantics when characterizing the tokens in *Stmts*. By integrating data semantics from the DFP task (i.e., 30 to *mem*, 40 to *temp*) and control semantics from the CFP task (i.e., `strcpy` involves size checks), PVDetector can encode tokens with these combined semantics. This comprehensive encoding is crucial for detecting vulnerabilities such as buffer overflows, as it reveals that code semantics (i.e., *mem* of 30 and *temp* of 40) align with the predefined vulnerability knowledge (i.e., *mem* < *temp*), thereby exposing the security issues.

Task Formulation. This formulation of the VKP task is identical to the DFP task and will not be described in detail here.

4.4 VSD

VCSG introduces semantics-cognizant dataflow to represent a variable experiencing value changes with multiple nodes. To understand different semantics for the same variable, we define the VSD task. Unlike the previous three tasks, the VSD task, formulated as (*Stmts*, [SEP], *Flow*), aims to predict the masked distinct names of one variable in *Stmts* given the *Flow* (where the variable is assigned distinct names). As shown in Figure 4(d), for a flow where *mem_num2* is derived from *mem_num1*, both originating from *mem_num*, the VSD task masks the two VCSG variable nodes in the statements, prompting the model to predict them as distinct. This enables the model to differentiate instances of the same variable during encoding. For the buffer overflow vulnerability in the sample function, the VSD task helps distinguish between the two instances of *mem_num* in line 6, capturing the correct dataflow from 30 (instead of 80) to *mem*, thereby facilitating accurate vulnerability detection.

Task Formulation. Let d_i indicate one semantic-cognizant dataflow, s_i denote its corresponding statements, and $T = \{t_1, t_2, \dots, t_n\}$ represent variables associated with d_i . This task takes $s_i || t_j$ ($t_j \in T$) as input and aims to minimize the loss function for predicting the masked token t_j , formulated as:

$$L_{VSD} = - \sum_{t_j \in T} \log(y_j). \quad (2)$$

Here, y_j is the predicted value for t_j , denoted by $\text{softmax}(O_{[t_j]}(s_i || t_j))$, where $O_{[t_j]}$ is the output value of token t_j in the final layer of the VSD task.

4.5 SP Conclusion

Since existing code-pretrained models are more targeted at sequential data, Semantics Pretraining is specifically designed to acquire various semantics from the structured VCSG. This approach inherently accounts for the hierarchical and non-linear nature of control and dataflow in code. Consequently, the produced token embeddings can simultaneously capture these features, providing a more accurate comprehension of code semantics, as shown in Figure 4.

5 Implementation

We have implemented a prototype of PVDetector. We utilize tree-sitter, a lightweight code parsing tool [78], to generate an AST for each function code. By recursively traversing the AST from the root node, we extract dataflow, control flow, and vulnerability relations to build the VCSG model. Note that the tree-sitter supports multiple programming languages, facilitating the extension of PVDetector to additional languages. This process involves handling AST identifiers and extracting vulnerability knowledge. Currently, PVDetector supports C, C++, Java, Python, PHP, and JavaScript. Figure 5 illustrates the training and testing phases of PVDetector.

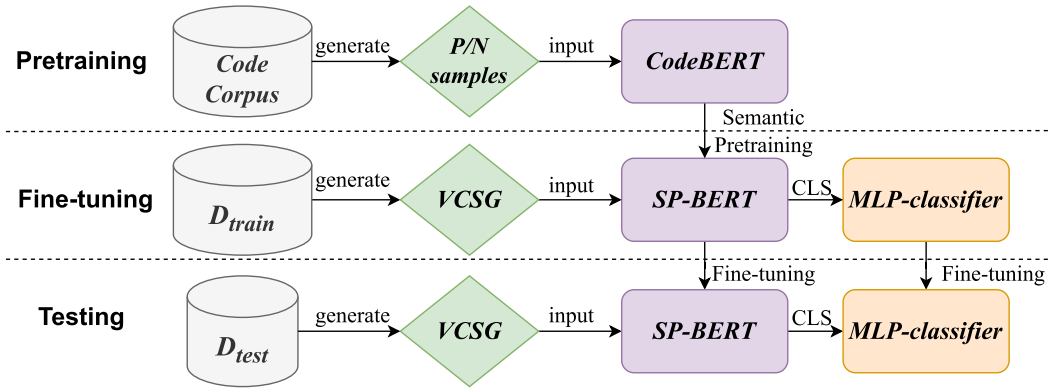


Fig. 5. The implementation of PVDetector.

During the pretraining phase, we conduct Semantics Pretraining based on the CodeBERT [23] model using the same code corpus (i.e., CodeSearchNet [30]) as CodeBERT. To support the four pretraining tasks proposed in Section 4, we traverse the VCSG nodes to generate the training samples. For each node, we create positive CFP (or VSD) samples by pairing the node with its children that have control dependencies (or distinct semantics). For data dependencies, DFP-positive samples are generated by pairing the node with its child and grandchild nodes. We exclude deeper descendants as their relationships can be inferred from multiple samples. For vulnerability relations, VKP-positive samples are formed by selecting all relevant nodes based on vulnerability knowledge. Meanwhile, negative samples are created by randomly selecting non-child nodes for each positive sample. To clearly illustrate the process of generating positive and negative samples, we use the VKP task as an example. First, we locate the statement containing vulnerability knowledge using the method described in Section 3.4.3 and generate the corresponding vulnerability knowledge. Then, the combination of this vulnerability knowledge and the statement forms a positive sample, while the combination of this vulnerability knowledge and other statements forms a negative sample. The same approach applies to the other three tasks. We designate the resulting trained model as SP-BERT, which serves as the foundation for fine-tuning.

During the fine-tuning phase, we fine-tune the SP-BERT with labeled vulnerability datasets to prove the effectiveness of PVDetector. Specifically, the [CLS] embedding of the SP-BERT will be input into an MLP-based binary classifier comprising two fully connected layers with dropouts. Cross-entropy loss is employed during this process. Subsequently, the optimized SP-BERT and classifier are utilized in the testing phase for the final vulnerability detection. Our code is available at <https://github.com/yoimiya-nlp/PVDetector>.

6 Evaluation

We evaluate PVDetector with the following **Research Questions (RQs)**:

- **RQ1:** *Is it effective in vulnerability detection (Section 6.2)?* Through this RQ, we aim to evaluate whether PVDetector outperforms existing SOTA vulnerability detection methods in terms of detection effectiveness.
- **RQ2:** *What is the runtime overhead on detection (Section 6.3)?* We aim to analyze the runtime overhead introduced by PVDetector and compare it against existing methods.

Table 2. The Statistics of Studied Datasets

Dataset	# Vul.	# Non-Vul.	# Total	% Vul.:Non-Vul.
PVData	3,244	5,228	8,472	0.621
PVData+	15,362	36,960	52,322	0.416
BigVul	10,900	177,736	188,636	0.061
7Projects	118	1,272	1,390	0.093
SARD	45,520	62,785	108,305	0.725

- RQ3: Can the various components in PVDetector effectively improve its vulnerability detection performance (Section 6.4)? To better understand the real contributions of each proposed component, we conduct ablation studies to assess their individual impacts.
- RQ4: How does it perform in real-world scenarios (Section 6.5)? We evaluate the real-world performance of PVDetector by applying it to seven real-world software projects, with the goal of assessing its practical utility in industrial vulnerability detection scenarios.
- RQ5: Can it generalize to multiple languages (Section 6.6)? While existing methods primarily focus on detecting vulnerabilities in C/C++ code, PVDetector supports multiple languages. Therefore, we further investigate its effectiveness in detecting multi-language vulnerabilities.
- RQ6: Can LLMs use PVDetector (mainly VCSG) to enhance their vulnerability detection capabilities (Section 6.7)? Given the strong understanding and reasoning capabilities of LLMs, we are also interested in whether incorporating VCSG as contextual information in prompts can enhance the vulnerability detection performance of LLMs. This investigation will determine whether VCSG has a broader application perspective.

6.1 Experimental Setup

Dataset. We aim to evaluate PVDetector’s ability to identify real-world vulnerabilities and its performance across various vulnerability categories. Existing datasets either record synthetic data [66] or record real data but lack category descriptions [9, 102]. Thus, we prepare a dataset, PVData, following VulDetector’s method [14]. Specifically, we collect vulnerable functions and their patched versions from Linux, OpenSSL, and Wireshark, and extract neighboring functions as normal samples. In total, PVData consists of 5,228 normal and 3,244 vulnerable functions across 43 CWEs, enabling a detailed assessment of sensitivity to specific vulnerability types.

We further enrich PVData with additional samples and evaluate PVDetector on five datasets to answer different questions: (1) PVData (RQ1), to measure performance across different CWE categories. (2) PVData+ (RQ1, RQ2, RQ3): It extends PVData with samples from commonly used datasets, Devign [102] (collected from four large, open source C projects: QEMU, FFmpeg, Wireshark, and the Linux Kernel) and Reveal [9] (collected from two open source C projects: Linux Debian Kernel and Chromium), to evaluate PVDetector in a large real-world dataset. After deduplication, PVData+ includes 15,362 vulnerable and 36,960 normal C/C++ functions. (3) Big-Vul (RQ1), a large C/C++ dataset to measure performance on unbalanced datasets [22], which is collected from 348 open source projects spanning 91 different vulnerability types from 2002 to 2019 [56]. To ensure a fair comparison, we evaluate PVDetector on the specific version’s Big-Vul filtered by SVuLD [56], and the previous methods are also re-evaluated on it. (4) Seven C/C++ projects (RQ4), to assess the detection performance of PVDetector in practical usage. (5) SARD (RQ5), a semi-synthetic dataset to measure performance across multiple programming languages. It includes 45,520 vulnerable and 62,785 normal functions, covering C/C++, Java, Python, PHP, and JavaScript. The statistics of all these five datasets are reported in Table 2.

SOTA Approaches. We compare PVDetector with 10 SOTA approaches. VulDeePecker is a pioneering DL-based approach [49]. SySeVR [48] extends VulDeePecker with data and control flow features from code snippets. Devign [102] uses CPG to extract syntactic and semantic features. Similarly, Reveal [9] employs Gated GNN to learn features from the CPG. VulCNN [93] treats PDG as an image and uses CNN for detection. SVulD [56] utilizes contrastive learning to differentiate between lexically similar but semantically different functions. AMPLE [90] simplifies code structure graphs to better capture relations between distant nodes. CasualVul [62] improves performance by removing spurious features learned by models, we select its best UnixCoder [26] model. LineVul [24] and DeepDFA [73] are described in Section 2.1.

Notably, all approaches based on pretrained models were pretrained on the identical dataset (i.e., CodeSearchNet). Furthermore, we ensure experimental fairness by implementing hash-based deduplication to guarantee that test data do not appear in either pretraining or fine-tuning datasets.

Configuration. We run models on a server configured with NVIDIA A100-PCIE-40GB GPU. All comparison methods are configured with their default models and parameters. Unless otherwise specified, we randomly split the data into 70% for training, 10% for validation, and 20% for testing. Due to the deduplication process, there is no overlap between the training and testing sets after dataset splitting. To ensure fairness, all methods are evaluated using the same training and testing sets. For highly imbalanced datasets, we perform undersampling [32] of the majority class during training, while maintaining the original vulnerable/non-vulnerable ratio during testing. We conduct the experiments five times and report the average results.

Metrics. We measure the effectiveness by four widely used metrics, i.e., Accuracy (A), Precision (P), Recall (R), and F1-score (F1), which are computed by the following equations:

$$A = \frac{TP + TN}{TP + TN + FP + FN}, \quad (3)$$

$$P = \frac{TP}{TP + FP}, \quad (4)$$

$$R = \frac{TP}{TP + FN}, \quad (5)$$

$$F1 = \frac{2 * P * R}{P + R}, \quad (6)$$

where **True Positive (TP)** denotes correctly identified vulnerable functions, **True Negative (TN)** denotes correctly predicted normal functions, FP denotes normal functions mistakenly identified as vulnerable, and **False Negative (FN)** denotes vulnerable functions mistakenly predicted as normal. These metrics facilitate a more comprehensive evaluation of vulnerability detection.

6.2 RQ1. Detection Performance

6.2.1 Performance on Real-World Dataset. We first compare these models on three real-world datasets. From the results reported in Table 3, we derive the following observations:

PVDetector Outperforms all These SOTA Methods. PVDetector demonstrates substantial improvements compared to all SOTA methods (i.e., 5.0–12.5% in precision, 0.2–9.7% in recall, and 3.0–15.1% in F1-score). Specifically, PVDetector achieves an improvement in the F1-score of 3.0%, 13.8%, and 15.1% on the PVDData, PVDData+, and Big-Vul datasets, respectively. These results indicate that PVDetector achieves superior performance in various vulnerability detection scenarios.

For previous SOTA methods, we also analyze the reasons why they could not achieve superior results. VulDeePecker can only detect vulnerabilities related to API calls, limiting its recall to 27.64% on PVDData+. Devign, Reveal, and AMPLE employ CPG to represent code but lack vulnerability awareness, struggling to learn vulnerability-related features. Although LineVul employs pretrained

Table 3. Comparison with SOTA Approaches on Real-World Datasets

Approach	PVData				PVData+				Big-Vul			
	A (%)	P (%)	R (%)	F1 (%)	A (%)	P (%)	R (%)	F1 (%)	A (%)	P (%)	R (%)	F1 (%)
VulDeePecker (NDSS18)	72.46	69.27	55.67	61.73	58.83	29.09	27.64	28.34	79.81	7.43	25.29	11.48
SySeVR (TDSC22)	83.20	80.04	77.13	78.56	67.07	44.46	47.23	45.80	84.43	11.36	29.47	16.39
Devign (NIPS19)	80.05	74.36	76.32	75.32	72.53	54.69	39.31	45.74	85.65	14.59	36.50	20.85
Reveal (TSE22)	88.21	87.18	82.59	84.82	72.29	52.39	65.10	58.06	85.36	18.07	51.71	26.78
VulCNN (ICSE22)	84.65	82.07	78.74	80.37	71.93	52.75	45.09	48.62	86.55	16.24	38.40	22.82
LineVul (MSR22)	89.18	88.14	84.21	86.13	76.70	59.31	66.65	62.76	82.72	18.17	66.73	28.57
SVulD (FSE23)	90.06	87.17	88.06	87.61	79.12	65.15	62.63	63.87	89.85	26.16	52.66	34.95
AMPLE (ICSE23)	91.76	91.35	87.65	89.46	78.09	64.32	57.54	60.74	89.23	23.94	49.62	32.30
CausalVul (ICSE24)	92.33	91.13	89.47	90.30	79.51	64.42	68.05	66.18	91.57	31.79	54.75	40.22
DeepDFA (ICSE24)	91.84	89.54	90.08	89.81	78.81	61.93	72.82	66.94	89.73	29.47	70.53	41.57
PVDetector	94.59	95.91	90.28	93.01	85.39	73.31	79.30	76.19	91.27	34.64	77.38	47.85

Bold indicates the best performance.

models, it does not exploit the unique structural characteristics inherent in the source code (i.e., 13.43% in F1 lower than PVDetector on PVData+). VulCNN treats PDG as an image, less intuitive than code semantics in data and control flow, leading to poor performance (i.e., 22.82% in F1 on Big-Vul). SVulD and CasualVul take a different perspective by leveraging contrastive learning and casual learning, respectively, to guide pretrained models in utilizing correct features for prediction. This approach allows them to achieve better results compared to previous methods (i.e., 3.13% and 5.44% in F1 over AMPLE on PVData+). However, their performance is limited by a lack of code structure exploration and vulnerability knowledge utilization. While DeepDFA demonstrates robust utilization of control and dataflow information, achieving notable recall rates (i.e., 70.53% on Big-Vul), its lack of vulnerability knowledge integration also leads to lower performance compared to PVDetector. PVDetector simultaneously captures both code structural semantics and vulnerability semantics while effectively incorporating these semantics into code representations through semantic pretraining, thereby achieving superior detection performance.

Performance Deterioration in Real-World Scenarios. As the dataset becomes more realistic, i.e., incorporating more complex vulnerabilities and exhibiting imbalance, the performance of these approaches declines sharply. For example, SySeVR's F1 drops from 78.56% on PVData to 45.8% on PVData+, where more vulnerabilities are included. On Big-Vul, which maintains a vulnerable ratio of 5.7%, the performance further drops to 16.39%. This is understandable since more vulnerabilities can manifest themselves in various forms and characteristics, making detection more challenging, especially in unbalanced datasets. Despite the difficulty, PVDetector still outperforms other approaches, improving recall by 9.7% and F1 by 15.1% over DeepDFA, demonstrating its superior ability to characterize the semantics of different vulnerabilities.

6.2.2 Performance for Different CWEs. We assess the models' sensitivity to different vulnerability types using PVData, which tracks CVE-CWE relationships. Figure 6 shows the results of the top 10 CWEs with the most samples. As we can see, the detection results vary significantly. Notably, PVDetector achieves high F1 for CWE119 (95.39%) and CWE415 (96.33%) but experiences a noticeable decrease for CWE399 (89.42%). This decrease is due to the underutilization of vulnerability information since these CWEs except CWE399 have corresponding entries in our knowledge base. Consequently, VCSG lacks adequate security semantics about CWE399, leading to reduced performance. Despite this, our approach outperforms other methods for each CWE type. For example, compared to the best SOTA results (CasualVul), PVDetector improves the F1 by 0.53–3.05% for these 10 CWEs, demonstrating its detection capabilities across different types of vulnerabilities.

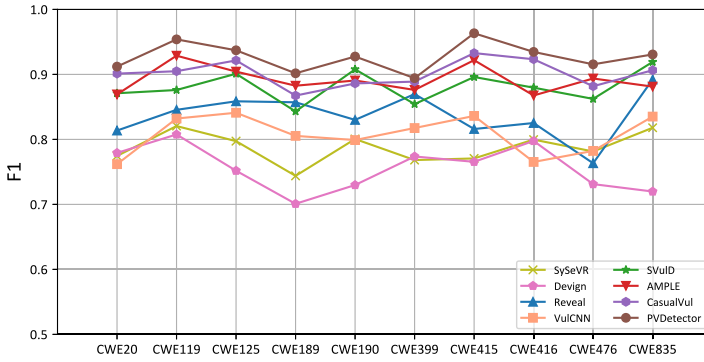


Fig. 6. Comparison of F1 (%) for different CWEs on PVDData.

6.2.3 Understanding the Learned Semantics. To further explore PVDetector’s effectiveness, particularly in capturing code and vulnerability semantics, we present a correlation matrix illustrating semantic similarity between VCSG nodes. Specifically, we derive the activation vectors of VCSG nodes from the final transformer layer in PVDetector and compute their semantic similarity using inner products. Figure 7 displays the matrix for the VCSG in Figure 2(b), with higher values indicating stronger associations. As expected, the correlation score between *temp1* and *temp2* reaches 0.82, reflecting their dataflow connection, as does the correlation score between *mem_num2* and *mem1*. Additionally, the correlation score between *temp2* and 40 is also high (0.47), showing that PVDetector can effectively capture long-range data dependencies.

For *TEMP_NUM* and *mem1* not connected in VCSG, their correlation is 0.51 instead of 0, implying that PVDetector captures their control semantics. Despite no direct data or control flow, *mem1* and *temp2* show a high correlation of 0.92, reflecting captured vulnerability dependencies and confirming the effectiveness of pretraining tasks. Additionally, *mem_num1* and *mem_num2*, originating from the same variable, show a correlation of 0.81. This is even lower than the 0.92 correlation between *mem1* and *temp2* with implicit dependencies, demonstrating that PVDetector effectively distinguishes the semantics of the same variable. Overall, the matrix confirms that PVDetector effectively learns code and vulnerability semantics, contributing to its capability in vulnerability detection.

Answer to RQ1. PVDetector outperforms all SOTA methods, achieving relative improvements in F1 score of 3.0%, 13.8%, and 15.1% over the best-performing existing method across three datasets. The visualized correlation matrices further demonstrate that PVDetector effectively captures accurate semantic representations of code and vulnerability.

6.3 RQ2. Runtime Overhead

Since training is a one-time cost, we focus on runtime overhead upon testing on the PVDData+ set of 10,465 samples. These functions range from 4 to 468 lines of code, with an average of 122. For the VCSGs, the average numbers of nodes and edges are 295.3 and 579.8. In comparison, the CPG graph, which also incorporates dataflow and control flow, contains an average of 793.2 nodes and 1679.4 edges, indicating a higher structural complexity than the VCSG model.

The testing process involves two main stages: (i) preprocessing to generate representations like VCSGs or CPGs from code, and (ii) classification to extract features and perform detection. To ensure a fair comparison of the two-stage runtime overhead, Figure 8 only compares the five existing methods which also include a preprocessing stage like PVDetector. As seen, PVDetector

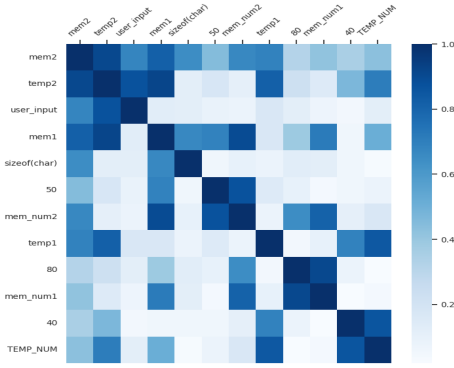


Fig. 7. Correlations matrix between VCSG nodes.

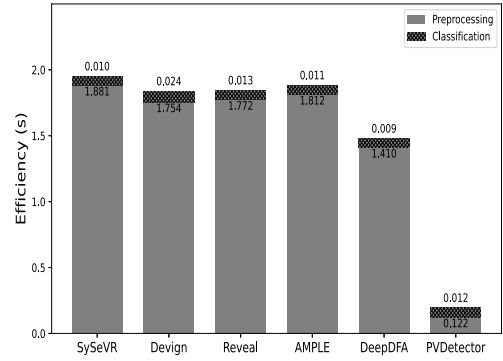


Fig. 8. Comparison of runtime overhead per sample.

processes 10,465 samples in 1,277 seconds for VCSG generation, averaging 0.122 seconds per sample. In addition, it takes 0.012 seconds per sample for classification, resulting in a total overhead of 0.134 seconds. This makes PVDetector 10.6 $\times$ faster than the SOTA method DeepDFA. PVDetector's efficiency stems from using the lightweight tree-sitter for source code parsing and targeting specific nodes in graphs. In contrast, SySeVR performs additional code slicing, resulting in preprocessing times of 1.881 seconds per sample. Other approaches use heavier tools like Joern and generate more complex models (e.g., CPG, CFG) leading to longer processing times.

Answer to RQ2. PVDetector demonstrates significantly higher runtime efficiency compared to existing preprocessing-based vulnerability detection methods, achieving 10.6 $\times$ faster than the SOTA method DeepDFA.

6.4 RQ3. Ablation Study

PVDetector comprises two key components, VCSG and SP. Meanwhile, VCSG is enhanced with the semantic-cognizant dataflow, condition-aware control flow, and vulnerability knowledge, representing the comprehensive code semantics. Therefore, we conduct the ablation experiment on the PVData+ dataset to measure the gain of each proposed component. First, we set up a baseline, which directly fine-tunes CodeBERT on the source code. Then, we systematically evaluate the contribution of each component in PVDetector by individually removing each component and assessing the resultant performance.

As reported in Table 4, the incorporation of VCSG and SP significantly enhanced the performance of PVDetector, resulting in F1-score improvements of 18.97% and 11.91%, respectively. These substantial gains underscore the critical importance of both the code semantics encapsulated in VCSG and the DL-based model's comprehension of code semantics facilitated by pretraining for effective vulnerability detection. Among the three dependency relationships in VCSG, semantic-cognizant dataflow provides the largest boost, yielding an 8.54% increase in the F1 score. This observation highlights the paramount importance of data dependencies in code semantics. Additionally, the condition-aware control flow and vulnerability knowledge relationships contribute to F1 score improvements of 5.34% and 6.45%, respectively, also demonstrating the importance of the three innovation dependencies in VCSG.

Table 4. Contribution of Each Proposed Component

Method	A (%)	P (%)	R (%)	F1 (%)
Baseline	71.93	52.54	48.69	50.54
w/o SP	79.53	66.15	62.50	64.28
w/o VCSG	75.38	58.63	55.89	57.22
w/o sc-df	81.07	68.13	67.17	67.65
w/o ca-cf	82.26	68.64	73.21	70.85
w/o vk	82.36	70.50	68.99	69.74
PVDetector	85.39	73.31	79.30	76.19

ca-cf, condition-aware control flow; sc-df, semantic-cognizant dataflow; vk, vulnerability knowledge. Bold indicates the best performance.

Answer to RQ3. Both VCSG and SP contribute significantly to the performance of PVDetector, resulting in F1 score improvements of 18.97% and 11.91%, respectively. Among the three types of dependencies in VCSG, the semantic-cognizant dataflow provides the greatest improvement, leading to an F1 increase of 8.54%.

6.5 RQ4. Performance in Real-World Scenarios

Although PVData+ contains real-world vulnerability samples, it does not fully reflect practical usage, i.e., using a well-trained model to detect vulnerabilities in new projects. To this end, we select seven software products (QEMU 2.7.0/8.0.0, FFmpeg 2.3.6/6.1.1, Curl 7.54.0, Django 4.0.6, and Eclipse 4.3.2) that differ in syntax and semantics from the PVData+ functions and contain unknown vulnerabilities absent in PVData+. For each product, we randomly sample 200 program files from core modules and leverage PVDetector, trained on PVData+, to detect vulnerabilities in these files. As shown in Figure 9, PVDetector demonstrates substantial improvements over SOTA methods, achieving gains of 19.1% and 19.6% in recall and F1-score, respectively. This demonstrates that PVDetector not only achieves superior detection performance on test datasets but also maintains effective vulnerability detection in real-world scenarios, thus proving its enhanced practicality compared to existing approaches. It is worth noting that we performed pretraining on CodeSearchNet, which contains multiple programming languages, including Java and Python. Therefore, PVDetector can understand both Python and Java code. Fine-tuning solely on PVData+ also allows the model to detect vulnerabilities in Python (Django) and Java (Eclipse) code.

Specifically, of the 108 vulnerabilities reported in NVD for these files, PVDetector successfully identified 45. In addition, by manually examining the other 52 suspected vulnerabilities predicted by PVDetector, we find that 10 were silently patched by vendors in the latest software versions, bringing the total number detected by PVDetector to 55. We list a few samples in Table 5.

Listing 1 shows a vulnerability in Curl 7.54.0 that has not been reported in NVD (the 5th row in Table 5). This vulnerability is due to the variable *left*, declared in line 3, not being properly initialized. PVDetector successfully detects this issue for two reasons. First, it identifies that *left* is used in line 8 without being assigned, by analyzing the data flows in the VCSG model. Second, leveraging the third vulnerability knowledge (i.e., Uninitialized Variable) in Table 1, PVDetector correctly infers that this function has a vulnerability.

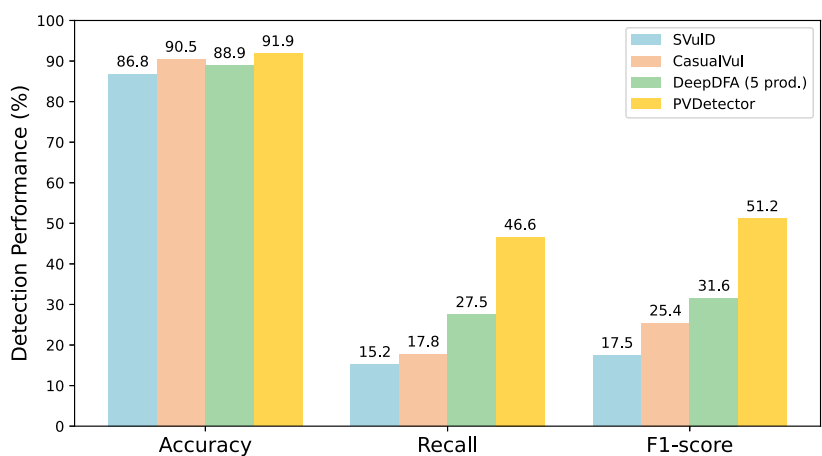


Fig. 9. Detection performance in real-world scenarios.

Table 5. Part of Vulnerabilities Detected by PVDetector in Real-World Scenarios

Target product	Language	CVE ID	Vulnerability type	File location	Status
QEMU 8.0.0	C	CVE-2023-40360	NULL Pointer Dereference (CWE-476)	.../ctrl.c	Report in NVD
		CVE-2024-3446	Double Free (CWE-415)	.../virtio-gpu.c	Report in NVD
FFmpeg 6.1.1	C	CVE-2023-51791	Buffer Overflow	.../jpegxl_parser.c	Report in NVD
		CVE-2024-22860	Integer Overflow (CWE-190)	.../jpegxl_anim_dec.c	Report in NVD
Curl 7.54.0	C	-	Variable uninitialized	.../ssh.c	No report
		-	Unused pointer not release	.../asyn-thread.c	No report
Django 4.0.6	Python	CVE-2023-23969	Resource Allocation (CWE770)	.../trans_real.py	Report in NVD
		CVE-2023-24580	Uncontrolled Resource Consumption (CWE400)	.../exception.py	Report in NVD
Eclipse 4.3.2	Java	CVE-2023-4218	Improper Restriction of XML (CWE611)	.../MacGenerator.java	Report in NVD

```
1 static CURLcode ssh_block_statemach(struct connectdata *conn, bool duringconnect) {
2     bool block;
3     time_t left;
4     ...
5     if(Curl_pgrsUpdate(conn))
6         return CURLE_ABORTED_BY_CALLBACK;
7     ...
8     if(left < 0) {
```

Listing 1. An uninitialized variable vulnerability in Curl.

Answer to RQ4. PVDetector demonstrates superior performance in real-world vulnerability detection scenarios, identifying 55 vulnerabilities across seven software products, better than existing SOTA methods. Notably, 10 of these vulnerabilities were silently patched by vendors in the latest versions and had not been reported in the CVE database.

6.6 RQ5. Performance for Multiple Languages

Since no real-world vulnerability dataset is available for multiple programming languages, we collect a multi-language vulnerability dataset from SARD and REEF [84]. This dataset includes 14,903 C/C++, 13,072 Java, 4,315 Python, 10,050 PHP, and 3,180 JavaScript vulnerable functions, alongside 20,366, 14,889, 7,679, 14,748, and 5,103 normal functions for these languages, respectively. We exclude

Table 6. Comparison of F1-Score (%) for Multiple Languages

Method	C/C++	Java	Python	PHP	JavaScript
SVulD	94.73	86.84	82.72	86.65	80.34
CasualVul	94.90	83.84	81.08	85.65	85.27
PVDetector	98.91	92.65	90.42	93.71	90.72

Bold indicates the best performance.

AMPLE and DeepDFA from the comparison because they rely on language-specific preprocessing procedures, which limit their generalization. Therefore, we select SVulD and CasualVul, the next best-performing approaches, for comparison. As reported in Table 6, PVDetector demonstrates robust performance across six languages, achieving an average F1-score of 93.28%, surpassing SVulD and CasualVul by margins of 6.02% and 6.13%, respectively. These results underscore PVDetector's exceptional capability in multi-language vulnerability detection.

Notably, compared to their performance in C/C++, all three approaches—SVulD, CasualVul, and PVDetector—exhibit performance degradation in other programming languages, with average decrements of 10.59%, 10.94%, and 7.03%, respectively. Given that these approaches are all based on pretrained models, this pattern indicates that the abundance of C/C++ language data in the pretraining corpus may have enhanced the transformer-based models' comprehension. Additionally, PVDetector demonstrates smaller performance degradation compared to SVulD and CasualVul. This observation yields two significant insights: first, it validates the multi-language generalization and universality of our proposed VCSG model; second, it indicates that SP facilitates better multi-language semantic understanding compared to the other conventional pretraining methods, accurately learning code semantics across different programming languages.

Furthermore, to provide a more comprehensive evaluation of PVDetector's vulnerability detection performance across different programming languages, we present the ROC curves in Figure 10. Specifically, for each dataset corresponding to the six programming languages, we randomly select 100 vulnerable samples and 100 non-vulnerable samples, and obtain the predicted probability scores from the PVDetector model. This sampling process is repeated five times to reduce the influence of randomness, and the averaged results are used for analysis. As shown in Figure 10, PVDetector consistently demonstrates excellent performance across all six programming languages, achieving AUC values exceeding 0.89. Notably, on the C/C++ dataset, PVDetector attains an AUC of 0.998, indicating near-perfect detection capability for this language. In contrast, the performance on Python is slightly lower, which may be attributed to a higher prevalence of complex vulnerabilities and the presence of noisy data that complicate the model's predictions. Additionally, we observe that when targeting a TP rate of approximately 90%, the corresponding FP rate remains around 10%. This balance enables PVDetector to detect the vast majority of vulnerabilities while incurring only modest verification overhead, thereby enhancing its practical applicability.

Answer to RQ5. PVDetector exhibits robust vulnerability detection performance across six programming languages, achieving an average F1 score that surpasses existing SOTA methods by 6.13%. This result highlights its superior capability for multi-language generalization.

6.7 RQ6. Improvement of VCSG on LLMs

To explore broader applications of VCSG, we incorporate VCSG into the prompt in the form of (*node1*, *node2*, *relation*) triplets to evaluate how the code semantics encoded in VCSG enhance LLMs' vulnerability detection capabilities. Our experimental evaluation is conducted using two open

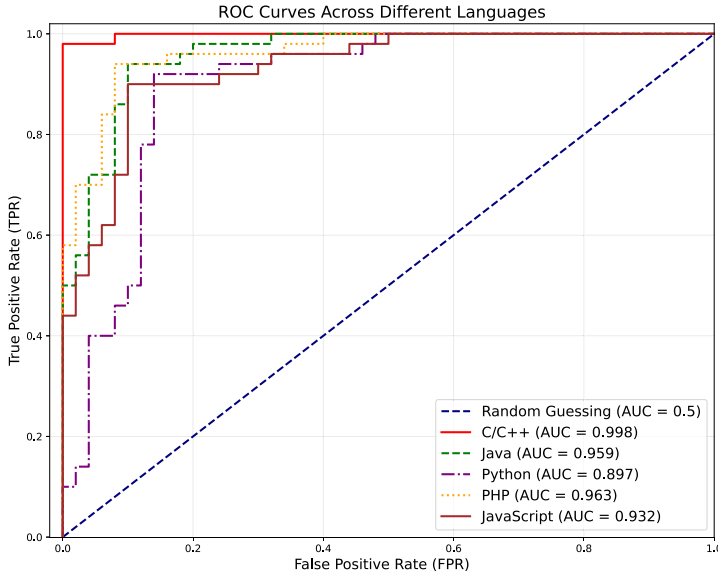


Fig. 10. ROC curves across different languages.

Table 7. Improvement of VCSG on LLMs

Method	A (%)	P (%)	R (%)	F1 (%)
llama-3.1-8b-it	68.20	52.43	48.02	50.13
+vcsg	74.47	62.57	57.92	60.15
codellama-7b-it	68.86	54.04	43.07	47.93
+vcsg	75.12	67.34	49.01	56.73
GPT-3.5	78.42	68.98	63.86	66.32
+vcsg	80.23	71.35	67.82	69.54
GPT-4o	77.43	63.05	77.72	69.62
+vcsg	80.56	68.10	78.22	72.81

Bold indicates the best performance.

source models, llama-3.1-8b-it and codellama-7b-it [77], along with two commercial models, GPT-3.5 and GPT-4o [59]. We access the GPT models through the OpenAI API, using gpt-3.5-turbo-0125 and gpt-4o-2024-08-06. Given computational and financial constraints, we randomly sample 202 vulnerable functions and 405 non-vulnerable functions from the PVDData+ dataset for evaluation. We conduct experiments to assess both the vulnerability detection performance of the original LLMs and the enhanced LLMs with VCSG-augmented prompts.

As shown in Table 7, incorporating VCSG into prompts significantly enhanced the vulnerability detection performance of LLMs, with average F1-score improvements of 9.41% and 3.16% for llama and GPT models, respectively. Notably, we observe that VCSG’s enhancement was more pronounced in llama models than in GPT models. This stems from GPT’s substantially larger parameter count, where its extensive internal knowledge may diminish the influence of external knowledge (VCSG in prompts) on its decision-making process. In conclusion, VCSG effectively acquires sufficient code semantics to enhance LLMs’ vulnerability detection capabilities.

Answer to RQ6. Experiments on four widely used open source and commercial LLMs demonstrate that the semantic information captured by VCSG indeed enhances the performance of LLM-based vulnerability detection, underscoring its broader applicability perspective.

7 Limitations and Future Work

Although PVDetector is effective in many ways, it has some limitations. First, the vulnerability knowledge base we prepared follows the CWE TOP 25 and contains 20 entries. This limited scope cannot cover the full range of vulnerability knowledge. Second, while PVDetector supports many languages and performs well on the SARD and REEF datasets, no multi-language real-world dataset is available to validate its capabilities in practical usage. Third, while PVDetector demonstrates high accuracy in vulnerability detection, it currently lacks the capability for vulnerability localization.

We plan to improve PVDetector in three directions. First, we will enhance knowledge formalization to better represent vulnerability semantics and expand the vulnerability knowledge base by examining more CWEs and CVEs. Second, we aim to create a real-world vulnerability dataset spanning multiple programming languages to validate PVDetector's capabilities in practical usage. Third, we plan to extend PVDetector to not only identify the presence of vulnerabilities but also pinpoint their exact line locations within the code, thereby providing more assistance.

8 Discussion

8.1 Threats to Validity

8.1.1 Internal Validity. The primary threats to internal validity in our study stem from potential data leakage and limitations in vulnerability knowledge expansion. We evaluate PVDetector on three widely used real-world datasets: Reveal, Devign, and Big-Vul. However, these datasets may contain duplicate samples [13], which can result in data leakage between the training and testing sets, potentially biasing the reported results. To address this concern and ensure the fairness of the evaluation, we apply hash-based deduplication to eliminate duplicates across training and test sets.

In addition, to facilitate scalable and reproducible vulnerability knowledge construction, we design an automated and standardized vulnerability knowledge generation and formalization method, as detailed in Section 3.4. This method allows future researchers to extend the vulnerability knowledge base with ease, thereby improving coverage across more diverse vulnerability types.

8.1.2 External Validity. Threats to external validity primarily concern the generalizability of our approach. Most existing vulnerability detection methods are evaluated exclusively on datasets derived from C/C++ code, limiting their applicability to other widely used programming languages. To overcome this limitation, we leverage tree-sitter's support for multi-language parsing and extend PVDetector to six popular programming languages, including Java, Python, PHP, and JavaScript. This significantly enhances the language generalizability of our method and makes it more applicable to real-world, heterogeneous codebases.

8.1.3 Construct Validity. Threats to construct validity are related to the choice of evaluation metrics. We employ four standard and commonly used metrics—Accuracy, Precision, Recall, and F1-score—to assess the overall detection performance of PVDetector. These metrics reflect both key dimensions of a practical vulnerability detection system: (1) its ability to identify as many true vulnerabilities as possible, which is essential for ensuring software security; and (2) its ability to minimize FPs, which can otherwise lead to substantial manual inspection costs. We believe that these metrics provide a reliable and balanced evaluation of PVDetector's detection capability.

8.1.4 Conclusion Validity. Threats to conclusion validity arise primarily from the representativeness of the test data and potential biases in the choice of baseline methods. To mitigate these concerns, we evaluate PVDetector on three commonly used public datasets—Devign, Reveal, and Big-Vul—and repeat all experiments five times to reduce the impact of randomness. Furthermore, we collect additional datasets to evaluate PVDetector’s performance across different CWE categories and programming languages, thus offering a more comprehensive assessment.

To minimize bias in baseline selection, we compare PVDetector against both classic and SOTA DLVD methods, including non-pretrained and pretrained methods. The selected baselines include foundational works as well as top-performing methods published in the past 2 years. This comprehensive comparison helps ensure that our conclusions are well grounded and reflect the relative strengths of our approach within the current research landscape.

8.2 Implications for Researchers and Practitioners

For researchers, the proposed semantic-cognizant dataflow and condition-aware control flow, which offer more precise code semantics, can be applied to a variety of code-related tasks such as code understanding [99], code summarization [7], and code repair [100]. Furthermore, our work demonstrates the feasibility of enhancing pretrained models’ ability to learn semantic information from code graph structures by designing novel pretraining tasks. This finding extends the research boundary of code pretraining models [23, 26]. The proposed approach is generalizable across different pretraining architectures and diverse types of code graph structures. The code representations generated by the pretrained models, which incorporate fused graph-structured semantics, can also serve as foundational representations for multi-modal learning and representation learning tasks.

For practitioners, our approach can serve as an enhancement module for static analysis tools, assisting developers in automatically detecting potential vulnerabilities before code submission, particularly within Continuous Integration workflows, where such tools are of practical value. Experimental results in Section 6 show that PVDetector achieves higher precision while maintaining a higher recall compared to existing methods. In other words, PVDetector is capable of identifying a greater number of potential vulnerabilities in the code while also maintaining a lower false-positive rate. This balance improves developer trust and reliance on the tool, thereby contributing to streamlined development workflows and enhanced code quality assurance mechanisms.

9 Related Work

Existing static vulnerability detection methods can be categorized into two main classes: code similarity-based methods and pattern-based methods.

9.1 Code Similarity-based Methods

Code similarity-based methods detect vulnerabilities by identifying cloned code, which falls into four types based on how the source code is utilized. Text-based methods [36, 38, 63, 74] directly analyze the source code. Token-based techniques [31, 36, 63] tokenize the source code according to lexical rules and then compare the token sequences. Tree-based methods [34, 39] parse the source code into ASTs and detect clones through tree matching. Graph-based methods [14, 35, 42, 80, 94] represent the source code with code structures such as CFGs, PDGs, or CPGs and compare graph structures. These methods can only detect vulnerabilities that match previously identified patterns. If a vulnerability is not based on code cloning, then detection methods relying on code similarity cannot be effectively utilized to recognize such a vulnerability.

9.2 Pattern-based Methods

Pattern-based methods can be categorized into four main classes: rule-based methods, ML-based methods, DL-based methods, and LLM-based methods.

Rule-based methods [4, 10, 67, 91] rely on manually defined vulnerability rules and characteristics from the security experts, which makes it difficult to detect the undefined vulnerabilities, thereby suffering high false negatives when encountering new vulnerabilities.

ML-based methods [25, 53, 54, 69, 85, 97] first extract static features from the source code, such as code complexity [70, 83], function calls [55], variable definitions [71], and code structures (CFG, AST) [96]. These features are then fed into the ML models, such as Decision Trees, SVMs, or MLPs, to detect vulnerabilities. However, these methods require manual feature engineering, and the extracted features are more syntactical rather than semantical.

DL-based methods [9, 45, 47–49, 56, 62, 90, 93, 102] have been the most popular methods in the past few years. Early DL-based vulnerability detection methods [49, 64] used simple DL models, such as LSTM and CNN, to generate code representations for vulnerability detection. However, these source-code-based methods yielded unsatisfactory results because they overlooked the unique logical structure inherent in code, which differs from natural language. To address this issue, some studies [9, 17, 90, 92, 93, 102] employed program analysis techniques to extract structural information containing both syntactic and semantic elements from the source code for vulnerability detection. These methods extract various structural features from the source code, such as ASTs, dataflow graphs, CFGs, and CPGs, and use DL models, particularly graph neural networks, to learn these code structural features for vulnerability detection. However, due to the inherent limitations of traditional DL models, such models often suffer from overfitting, performing well on one dataset while experiencing significant performance degradation on others. With the development of pretrained models like BERT [19] and the transformer architecture [82], code pretrained models, such as CodeBERT [23], GraphCodeBERT [27], CodeT5 [86], and UnixCoder [26], have emerged, incorporating extensive code semantic information. These pretrained models have demonstrated excellent performance in tasks like code summarization and code search [2, 50]. Consequently, methods like SVulD [56] and CasualVul [62], which apply pretrained code models to vulnerability detection, have also shown promising results. LineVul [24] and DeepDFA [73] introduced innovations in the byte pair encoding [68] for the code vocabulary, but they did not address the challenge of effectively learning code graph structure information with pretrained models. Additionally, these methods fail to capture the unique vulnerability semantics inherent in the source code, which adversely affects the performance of vulnerability detection.

LLM-based methods have recently emerged and show significant research potential. Research in the field of DL reveals that when the scale of pretrained language models (in terms of parameters, data, and computation) surpasses a certain threshold, an emergence phenomenon occurs [88]. That is, the model suddenly acquires abilities that are completely absent in smaller models, with these improvements manifesting as nonlinear leaps. Based on this phenomenon, commercial and open source LLMs, such as GPT [59], Llama [77], and DeepSeek [18], have been developed. These models possess powerful comprehension and logical reasoning capabilities, allowing them to solve specific tasks through prompt learning [61] and context learning [6] without the need for fine-tuning. With the emergence of these LLMs, several studies [51, 52, 57, 76] employ chain-of-thought [89] or prompt learning methods to utilize their powerful reasoning capabilities for vulnerability detection. However, research [79] indicates that despite their significant potential, LLMs cannot yet serve as reliable vulnerability detection tools. LLM-based vulnerability detection methods currently face challenges, including non-deterministic detection results and substantial computational costs [79], which necessitate targeted solutions through innovative future research.

10 Conclusion

In this article, we propose PVDetector, a novel DLVD approach that leverages rich code semantics, domain-specific vulnerability knowledge, and pretrained representations to enhance detection capabilities. We develop a VCSG model to accurately distinguish variable semantics and characterize control flow, dataflow, and vulnerability relations. Additionally, we design four pretraining tasks, DFP, CFP, VKP, and VSD, to learn informative code semantics from the VCSG model. Our evaluation on several real-world datasets demonstrates that PVDetector outperforms 10 SOTA approaches. From 7 software products, PVDetector identifies 10 previously unreported vulnerabilities that are absent from the NVD. Furthermore, we validate PVDetector's effectiveness across multiple programming languages and verify that VCSG can also improve LLMs' vulnerability detection capabilities.

References

- [1] Rakesh Agrawal, Tomasz Imielinski, and Arun N. Swami. 1993. Mining association rules between sets of items in large databases. In *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*. ACM Press, 207–216.
- [2] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT '21)*. Association for Computational Linguistics, 2655–2668.
- [3] Safwan Mahmood Al-Selwi, Mohd Fadzil Hassan, Said Jadid Abdulkadir, and Amgad Muneer. 2023. LSTM inefficiency in long-term dependencies regression problems. *J. Adv. Res. Appl. Sci. Eng. Technol.* 30, 3 (2023), 16–31.
- [4] Michael Backes, Konrad Rieck, Malte Skoruppa, Ben Stock, and Fabian Yamaguchi. 2017. Efficient and flexible discovery of PHP application vulnerabilities. In *2017 IEEE European Symposium on Security and Privacy (EuroS&P '17)*. IEEE, 334–349.
- [5] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2019. Coverage-based greybox fuzzing as Markov cha. *IEEE Trans. Softw. Eng.* 45, 5 (2019), 489–506.
- [6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020 (NeurIPS '20)*.
- [7] Liuwen Cao, Hongkui He, Hailin Huang, Jiexin Wang, and Yi Cai. 2025. Rethinking-based code summarization with chain of comments. In *Proceedings of the 31st International Conference on Computational Linguistics*, 3043–3056.
- [8] Sadeqh Bamohabbat Chafjiri, Phil Legg, Jun Hong, and Michail-Antisthenis I. Tsompanas. 2024. Vulnerability detection through machine learning-based fuzzing: A systematic review. *Comput. Secur.* 143 (2024), 103903. DOI : <https://doi.org/10.1016/j.cose.2024.103903>
- [9] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep learning based vulnerability detection: Are we there yet? *IEEE Trans. Softw. Eng.* 48, 9 (2022), 3280–3296.
- [10] Checkmarx. 2024. Checkmarx—Application Security. Retrieved February 3, 2024 from <https://www.checkmarx.com/>
- [11] Xiao Cheng, Haoyu Wang, Jiayi Hua, Miao Zhang, Guoai Xu, Li Yi, and Yulei Sui. 2019. Static detection of control-flow-related vulnerabilities using graph embedding. In *2019 24th International Conference on Engineering of Complex Computer Systems (ICECCS '19)*. IEEE, 41–50.
- [12] Xiao Cheng, Guanqin Zhang, Haoyu Wang, and Yulei Sui. 2022. Path-sensitive code embedding via contrastive learning for software vulnerability detection. In *31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '22)*. ACM, 519–531.
- [13] Roland Croft, M. Ali Babar, and M. Mehdi Kholoosi. 2023. Data quality for software vulnerability datasets. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE '23)*. IEEE, 121–133.
- [14] Lei Cui, Zhiyu Hao, Yang Jiao, Haiqiang Fei, and Xiaochun Yun. 2021. VulDetector: Detecting vulnerabilities using weighted feature graph comparison. *IEEE Trans. Inf. Forensics Secur.* 16 (2021), 2004–2017.
- [15] CVE. 2024. CVE—Common Vulnerabilities and Exposures. Retrieved February 3, 2024 from <https://cve.mitre.org/>
- [16] CWE. 2024. CWE—Common Weakness Enumeration. Retrieved February 3, 2024 from <https://cwe.mitre.org/top25/>
- [17] Hoa Khanh Dam, Trang Pham, Shien Wee Ng, Truyen Tran, John Grundy, Aditya Ghose, Taeksu Kim, and Chul-Joo Kim. 2018. A deep tree-based model for software defect prediction. arXiv:1802.00921. Retrieved from <https://arxiv.org/abs/1802.00921>

- [18] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, et al. 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948. Retrieved from <https://arxiv.org/abs/2501.12948>
- [19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT '19)* (Long and Short Papers). Association for Computational Linguistics, 4171–4186.
- [20] Xiaoning Du, Bihuan Chen, Yuekang Li, Jianmin Guo, Yaqin Zhou, Yang Liu, and Yu Jiang. 2019. Leopard: Identifying vulnerable code for vulnerability assessment through program metrics. In *Proceedings of the 41st International Conference on Software Engineering (ICSE '19)*. IEEE/ACM, 60–71.
- [21] Xu Duan, Jingzheng Wu, Shouling Ji, Zhiqing Rui, Tianyue Luo, Mutian Yang, and Yanjun Wu. 2019. VulSniper: Focus your attention to shoot fine-grained vulnerabilities. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, 4665–4671.
- [22] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ code vulnerability dataset with code changes and CVE summaries. In *17th International Conference on Mining Software Repositories (MSR '20)*. ACM, 508–512.
- [23] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics (EMNLP '20)*. Association for Computational Linguistics, 1536–1547. DOI : <https://doi.org/10.18653/v1/2020.findings-emnlp.139>
- [24] Michael Fu and Chakkril Tantithamthavorn. 2022. LineVul: A transformer-based line-level vulnerability prediction. In *19th IEEE/ACM International Conference on Mining Software Repositories (MSR '22)*. ACM, 608–620. DOI : <https://doi.org/10.1145/3524842.3528452>
- [25] Tom Ganz, Erik Imgrund, Martin Härterich, and Konrad Rieck. 2023. PAVUDI: Patch-based vulnerability discovery using machine learning. In *Annual Computer Security Applications Conference (ACSAC '23)*. ACM, 704–717.
- [26] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 7212–7225.
- [27] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2021. GraphCodeBERT: Pre-training code representations with data flow. In *9th International Conference on Learning Representations (ICLR '21)*.
- [28] Yijia Guo, Yuanwei Hou, Yongle Hao, and Wenjue Xu. 2021. Modeling and analysis of cyberspace threat sources based on vulnerabilities. In *Proceedings of the 14th International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing (IMIS '20)*. Springer, 292–303.
- [29] Mamoon Humayun, Mahmood Niazi, N. Z. Jhanjhi, Mohammad Alshayeb, and Sajjad Mahmood. 2020. Cyber security threats and vulnerabilities: A systematic mapping study. *Arab. J. Sci. Eng.* 45 (2020), 3171–3189.
- [30] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet challenge: Evaluating the state of semantic code search. arXiv:1909.09436. Retrieved from <http://arxiv.org/abs/1909.09436>
- [31] Jiyong Jang, Maverick Woo, and David Brumley. 2012. ReDeBug: Finding unpatched code clones in entire OS distributions. *Usenix* 37, 6 (2012).
- [32] Nathalie Japkowicz. 2000. The class imbalance problem: Significance and strategies. In *Proceedings of the 56th International Conference on Artificial Intelligence*, 111–117.
- [33] Bo Jiang, Ye Liu, and W. K. Chan. 2018. ContractFuzzer: Fuzzing smart contracts for vulnerability detection. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE '18)*. ACM, 259–269.
- [34] Lingxiao Jiang, Ghassan Mishserghi, Zhendong Su, and Stéphane Glondu. 2007. DECKARD: Scalable and accurate tree-based detection of code clones. In *29th International Conference on Software Engineering (ICSE '07)*. IEEE Computer Society, 96–105.
- [35] Andrew Johnson, Lucas Waye, Scott Moore, and Stephen Chong. 2015. Exploring and enforcing security guarantees via program dependence graphs. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 291–302.
- [36] J. Howard Johnson. 1993. Identifying redundancy in source code using fingerprints. In *Proceedings of the 1993 Conference of the Centre for Advanced Studies on Collaborative Research*. IBM, 171–183.
- [37] Nenad Jovanovic, Christopher Krügel, and Engin Kirda. 2006. Pixy: A static analysis tool for detecting web application vulnerabilities (short paper). In *2006 IEEE Symposium on Security and Privacy (S&P '06)*. IEEE Computer Society, 258–263.

- [38] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. VUDDY: A scalable approach for vulnerable code clone discovery. In *2017 IEEE Symposium on Security and Privacy (SP '17)*. IEEE Computer Society, 595–614.
- [39] Rainer Koschke, Raimar Falke, and Pierre Frenzel. 2006. Clone detection using abstract syntax suffix trees. In *13th Working Conference on Reverse Engineering (WCRE '06)*. IEEE Computer Society, 253–262.
- [40] Jiayuan Li, Lei Cui, Jie Zhang, Haiqiang Fei, Yu Chen, and Hongsong Zhu. 2025. Steering large language models for vulnerability detection. In *2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP '25)*. IEEE, 1–5.
- [41] Jiayuan Li, Lei Cui, Sen Zhao, Yun Yang, Lun Li, and Hongsong Zhu. 2025. CLVeR: Multi-modal contrastive learning for vulnerability code representation. In *Findings of the Association for Computational Linguistics (ACL '25)*. Association for Computational Linguistics, 7940–7951.
- [42] Jingyue Li and Michael D. Ernst. 2012. CBCD: Cloned buggy code detector. In *34th International Conference on Software Engineering (ICSE '12)*. IEEE Computer Society, 310–320.
- [43] Tuo Li, Jia-Ju Bai, Yulei Sui, and Shi-Min Hu. 2022. Path-sensitive and alias-aware tpestate analysis for detecting OS bugs. In *27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. ACM, 859–872.
- [44] Tuo Li, Jia-Ju Bai, Yulei Sui, and Shi-Min Hu. 2024. SPATA: Effective OS bug detection with summary-based, alias-aware, and path-sensitive tpestate analysis. *ACM Trans. Comput. Syst.* 42, 3-4 (2024), 1–40.
- [45] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2021. Vulnerability detection with fine-grained interpretations. In *29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. ACM, 292–303.
- [46] Yuekang Li, Yinxing Xue, Hongxu Chen, Xiuheng Wu, Cen Zhang, Xiaofei Xie, Haijun Wang, and Yang Liu. 2019. Cerebro: Context-aware adaptive fuzzing for effective vulnerability detection. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/SIGSOFT FSE '19)*. ACM, 533–544.
- [47] Zhen Li, Deqing Zou, Shouhuai Xu, Zhaoxuan Chen, Yawei Zhu, and Hai Jin. 2022. VulDeeLocator: A deep learning-based fine-grained vulnerability detector. *IEEE Trans. Dependable Secur. Comput.* 19, 4 (2022), 2821–2837.
- [48] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2022. SySeVR: A framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secur. Comput.* 19, 4 (2022), 2244–2258.
- [49] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. VulDeeP-ecker: A deep learning-based system for vulnerability detection. In *25th Annual Network and Distributed System Security Symposium*.
- [50] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. 2024. GraphCoder: Enhancing repository-level code completion via coarse-to-fine retrieval based on code context graph. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 570–581.
- [51] Guilong Lu, Xiaolin Ju, Xiang Chen, Wenlong Pei, and Zhilong Cai. 2024. GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning. *J. Syst. Softw.* 212 (2024), 112031.
- [52] Qiheng Mao, Zhenhao Li, Xing Hu, Kui Liu, Xin Xia, and Jianling Sun. 2024. Towards effectively detecting and explaining vulnerabilities using large language models. arXiv:2406.09701. Retrieved from <https://arxiv.org/abs/2406.09701>
- [53] Tina Marjanov, Ivan Pashchenko, and Fabio Massacci. 2022. Machine learning for source code vulnerability detection: What works and what isn't there yet. *IEEE Secur. Priv.* 20, 5 (2022), 60–76.
- [54] Kollin Napier, Tanmay Bhowmik, and Shaowei Wang. 2023. An empirical study of text-based machine learning models for vulnerability detection. *Empir. Softw. Eng.* 28, 2 (2023), 38.
- [55] Stephan Neuhaus, Thomas Zimmermann, Christian Holler, and Andreas Zeller. 2007. Predicting vulnerable software components. In *Proceedings of the 2007 ACM Conference on Computer and Communications Security (CCS '07)*. ACM, 529–540.
- [56] Chao Ni, Xin Yin, Kaiwen Yang, Dehai Zhao, Zhenchang Xing, and Xin Xia. 2023. Distinguishing look-alike innocent and vulnerable code by subtle semantic representation learning and explanation. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. ACM, 1611–1622.
- [57] Yu Nong, Mohammed Aldeen, Long Cheng, Hongxin Hu, Feng Chen, and Haipeng Cai. 2024. Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities. arXiv:2402.17230. Retrieved from <https://arxiv.org/abs/2402.17230>
- [58] NVD. 2024. National Vulnerability Database (NVD). Retrieved February 3, 2024 from <https://nvd.nist.gov/>
- [59] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774. Retrieved from <https://arxiv.org/abs/2303.08774>
- [60] Tushar P. Parikh and Ashok R. Patel. 2017. Cyber security: Study on attack, threat, vulnerability. *Int. J. Res. Mod. Eng. Emerg. Technol.* 5 (2017), 1–7.

- [61] Fabio Petroni, Patrick Lewis, Aleksandra Piktus, Tim Rocktäschel, Yuxiang Wu, Alexander H. Miller, and Sebastian Riedel. 2020. How context affects language models' factual predictions. In *Conference on Automated Knowledge Base Construction (AKBC '20)*.
- [62] Md Mahbubur Rahman, Ira Ceka, Chengzhi Mao, Saikat Chakraborty, Baishakhi Ray, and Wei Le. 2024. Towards causal deep learning for vulnerability detection. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24)*. ACM, Article 153, 1–11.
- [63] Chanchal Kumar Roy and James R. Cordy. 2008. NICAD: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization. In *The 16th IEEE International Conference on Program Comprehension (ICPC '08)*. IEEE Computer Society, 172–181.
- [64] Rebecca L. Russell, Louis Y. Kim, Lei H. Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul M. Ellingwood, and Marc W. McConley. 2018. Automated vulnerability detection in source code using deep representation learning. In *17th IEEE International Conference on Machine Learning and Applications*. IEEE, 757–762.
- [65] Hitesh Sajani, Vaibhav Saini, Jeffrey Svajlenko, Chanchal K. Roy, and Cristina V. Lopes. 2016. SourcererCC: Scaling code clone detection to big-code. In *Proceedings of the 38th International Conference on Software Engineering (ICSE '19)*. ACM, 1157–1168.
- [66] SARD. 2024. SARD. Retrieved February 3, 2024 from <https://samate.nist.gov/SARD/>
- [67] Inc. Secure Software. 2024. Rough Auditing Tool for Security (RATS). Retrieved February 3, 2024 from <https://github.com/wireghoul/rats>
- [68] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL '16)*. The Association for Computer Linguistics, 1715–1725.
- [69] Lwin Khin Shar, Lionel C. Briand, and Hee Beng Kuan Tan. 2015. Web application vulnerability prediction using hybrid program analysis and machine learning. *IEEE Trans. Dependable Secur. Comput.* 12, 6 (2015), 688–707.
- [70] Yonghee Shin, Andrew Meneely, Laurie A. Williams, and Jason A. Osborne. 2011. Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities. *IEEE Trans. Softw. Eng.* 37, 6 (2011), 772–787.
- [71] Grzegorz Siewruk and Wojciech Mazurczyk. 2021. Context-aware software vulnerability classification using machine learning. *IEEE Access* 9 (2021), 88852–88867.
- [72] Johannes Späth, Karim Ali, and Eric Bodden. 2017. IDE^{al}: Efficient and precise alias-aware dataflow analysis. *Proc. ACM Program. Lang.* 1, OOPSLA (2017), Article 99, 1–27.
- [73] Benjamin Steenhoeck, Hongyang Gao, and Wei Le. 2024. Dataflow analysis-inspired deep learning for efficient vulnerability detection. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24)*. ACM, Article 16, 1–13.
- [74] Hao Sun, Lei Cui, Lun Li, Zhenquan Ding, Zhiyu Hao, Jiancong Cui, and Peng Liu. 2021. VDSimilar: Vulnerability detection based on code similarity of vulnerabilities and patches. *Comput. Secur.* 110 (2021), 102417.
- [75] Yizhen Sun, Dandan Lin, Hong Song, Minjia Yan, and Linjing Cao. 2020. A method to construct vulnerability knowledge graph based on heterogeneous data. In *16th International Conference on Mobility, Sensing and Networking (MSN '20)*. IEEE, 740–745.
- [76] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Wei Ma, Lyuye Zhang, Miaolei Shi, and Yang Liu. 2024. LLM4Vuln: A unified evaluation framework for decoupling and enhancing LLMs' vulnerability reasoning. arXiv:2401.16185. Retrieved from <https://arxiv.org/abs/2401.16185>
- [77] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. LLaMA: Open and efficient foundation language models. arXiv:2302.13971. Retrieved from <https://arxiv.org/abs/2302.13971>
- [78] tree-sitter. [n.d.]. tree-sitter. Retrieved February 3, 2024 from <https://tree-sitter.github.io/tree-sitter/>
- [79] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse K. Coskun, and Gianluca Stringhini. 2024. LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks. In *IEEE Symposium on Security and Privacy (SP '24)*. IEEE, 862–880.
- [80] Ganesha Upadhyaya and Hridesh Rajan. 2018. On accelerating source code analysis at massive scale. *IEEE Trans. Softw. Eng.* 44, 7 (2018), 669–688.
- [81] Julien Vanegue and Shuvendu K. Lahiri. 2013. Towards practical reactive security audit using extended static checkers. In *2013 IEEE Symposium on Security and Privacy (SP '13)*. IEEE Computer Society, 33–47.
- [82] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems*, 5998–6008.
- [83] James Walden, Jeff Stuckman, and Riccardo Scandariato. 2014. Predicting vulnerable components: Software metrics vs text mining. In *25th IEEE International Symposium on Software Reliability Engineering (ISSRE '14)*. IEEE Computer Society, 23–33.

- [84] Chaozheng Wang, Zongjie Li, Yun Peng, Shuzheng Gao, Sirong Chen, Shuai Wang, Cuiyun Gao, and Michael R. Lyu. 2023. REEF: A framework for collecting real-world vulnerabilities and fixes. In *38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 1952–1962. DOI: <https://doi.org/10.1109/ASE56229.2023.00199>
- [85] Song Wang, Taiyue Liu, and Lin Tan. 2016. Automatically learning semantic features for defect prediction. In *Proceedings of the 38th International Conference on Software Engineering*, 297–308.
- [86] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP '21)*. Association for Computational Linguistics, 8696–8708.
- [87] Yongfu Wang, Ying Zhou, Xiaohai Zou, Quanqiang Miao, and Wei Wang. 2020. The analysis method of security vulnerability based on the knowledge graph. In *The 10th International Conference on Communication and Network Security (ICCNS '20)*. ACM, 135–145.
- [88] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022. Emergent abilities of large language models. *Trans. Mach. Learn. Res* 2022 (2022).
- [89] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed. H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022 (NeurIPS '22)*.
- [90] Xin-Cheng Wen, Yupan Chen, Cuiyun Gao, Hongyu Zhang, Jie M. Zhang, and Qing Liao. 2023. Vulnerability detection with graph simplification and enhanced graph representation learning. In *45th IEEE/ACM International Conference on Software Engineering (ICSE '23)*. IEEE, 2275–2286.
- [91] David A. Wheeler. 2024. Flawfinder—A Static Analysis Tool for Finding Security Vulnerabilities in C/C++ Source Code. Retrieved February 3, 2024 from <https://dwheeler.com/flawfinder/>
- [92] Hongjun Wu, Zhuo Zhang, Shangwen Wang, Yan Lei, Bo Lin, Yihao Qin, Haoyu Zhang, and Xiaoguang Mao. 2021. Peculiar: Smart contract vulnerability detection based on crucial data flow graph and pre-training techniques. In *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE '21)*. IEEE, 378–389.
- [93] Yueming Wu, Deqing Zou, Shihan Dou, Wei Yang, Duo Xu, and Hai Jin. 2022. VulCNN: An image-inspired scalable vulnerability detection system. In *44th IEEE/ACM 44th International Conference on Software Engineering (ICSE '22)*. ACM, 2365–2376. DOI: <https://doi.org/10.1145/3510003.3510229>
- [94] Yang Xiao, Bihuan Chen, Chendong Yu, Zhengzi Xu, Zimu Yuan, Feng Li, Binghong Liu, Yang Liu, Wei Huo, Wei Zou, and Wenchang Shi. 2020. MVP: Detecting vulnerabilities using patch-enhanced vulnerability signatures. In *29th USENIX Security Symposium (USENIX Security '20)*. USENIX Association, 1165–1182.
- [95] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and discovering vulnerabilities with code property graphs. In *2014 IEEE Symposium on Security and Privacy (SP '14)*. IEEE Computer Society, 590–604.
- [96] Fabian Yamaguchi, Markus Lottmann, and Konrad Rieck. 2012. Generalized vulnerability extrapolation using abstract syntax trees. In *28th Annual Computer Security Applications Conference (ACSAC '12)*. ACM, 359–368.
- [97] Fabian Yamaguchi, Alwin Maier, Hugo Gascon, and Konrad Rieck. 2015. Automatic inference of search patterns for taint-style vulnerabilities. In *2015 IEEE Symposium on Security and Privacy (SP '15)*. IEEE Computer Society, 797–812.
- [98] Fabian Yamaguchi, Christian Wressnegger, Hugo Gascon, and Konrad Rieck. 2013. Chucky: exposing missing checks in source code for vulnerability discovery. In *2013 ACM SIGSAC Conference on Computer and Communications Security (CCS '13)*. ACM, 499–510.
- [99] Weixiang Yan, Haitian Liu, Yunkun Wang, Yunzhe Li, Qian Chen, Wen Wang, Tingyu Lin, Weishan Zhao, Li Zhu, Hari Sundaram, et al. 2024. CodeScope: An execution-based multilingual multitask multidimensional benchmark for evaluating LLMs on code understanding and generation. In *62nd Annual Meeting of the Association for Computational Linguistics (ACL '24)*. Association for Computational Linguistics (ACL), 5511–5558.
- [100] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. 2024. Thinkrepair: Self-directed automated program repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 1274–1286.
- [101] Minmin Zhou, Jinfu Chen, Yisong Liu, Hilary Ackah-Arthur, Shujie Chen, Qingchen Zhang, and Zhifeng Zeng. 2019. A method for software vulnerability detection based on improved control flow graph. *Wuhan Univ. J. Nat. Sci.* 24, 2 (2019), 149–160.
- [102] Yaqin Zhou, Shangqing Liu, Jing Kai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Annual Conference on Neural Information Processing Systems*, 10197–10207.
- [103] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. 2019. μ VulDeePecker: A deep learning-based system for multiclass vulnerability detection. *IEEE Trans. Dependable Secur. Comput.* 18, 5 (2019), 2224–2236.

Received 1 March 2025; revised 6 September 2025; accepted 9 September 2025